Contents

1 Template **2**

2 Data Structures **2**
2.1 Segment Tree . . . 2
2.2 Merge Sort Tree . . . 2
2.3 Segment Tree Recursive . . . 2
2.4 Lazy . . . 2
2.5 Persistent Segment Tree . . . 3
2.6 Dynamic Segment Tree . . . 3
2.7 Sparse Table . . . 3
2.8 Treap . . . 3
2.9 BIT . . . 5
2.10 DSU . . . 5
2.11 DSU with Rollback . . . 5
2.12 Dynamic Connectivity . . . 6
2.13 Trie . . . 6
2.14 Mo's Algorithm . . . 6

3 Graphs **6**
3.1 2-SAT . . . 6
3.2 Block Cut Tree . . . 7
3.3 Bellman-Ford Cycle . . . 7
3.4 Bellman-Ford Path . . . 7
3.5 Bipartite Check . . . 7
3.6 Bridges . . . 7
3.7 Bridge Tree . . . 8
3.8 Dijkstra . . . 8
3.9 Dinic . . . 8
3.10 Euler Path Directed . . . 9
3.11 Euler Path Undirected . . . 9
3.12 Flood Fill . . . 9
3.13 Floyd-Warshall . . . 9
3.14 Kruskal . . . 9
3.15 Maximum Clique . . . 9
3.16 MCMF . . . 10
3.17 Prims . . . 10
3.18 SCC Kosaraju . . . 10
3.19 State Space Graph . . . 11
3.20 Topological Sort . . . 11
3.21 Virtual Tree . . . 11

4 Trees **11**
4.1 Centroid Decomposition . . . 11
4.2 Link Cut Tree . . . 11
4.3 Tree Diameter . . . 12
4.4 Euler Tour . . . 12
4.5 Heavy Light Decomposition . . . 12
4.6 LCA . . . 13
4.7 Min/Max Weight on Path . . . 13
4.8 Intersection Path . . . 13
4.9 Union Path . . . 13
4.10 Prufer Code Build . . . 14
4.11 Prufer Code Restore . . . 14
4.12 Tree Rerooting . . . 14
4.13 Tree Matching . . . 14
4.14 Rooted Tree Hashing . . . 14
4.15 Tree Isomorphism . . . 15

5 Number Theory **15**
5.1 Euler Totient . . . 15
5.2 Mobius Function . . . 15
5.3 Chinese Reminder Theorem . . . 15
5.4 Binpow . . . 15

6 Combinatorics **15**
6.1 nCr . . . 15

7 Math **15**
7.1 FFT . . . 15
7.2 FWDT . . . 15
7.3 NTT . . . 16
7.4 FFT Any Mod . . . 16
7.5 Or Convolution . . . 16
7.6 Xor Basis . . . 16
7.7 Matrix . . . 16
7.8 Gauss . . . 17
7.9 Gauss MOD . . . 17

8 Strings **17**
8.1 Phi Function . . . 17
8.2 Z Function . . . 17
8.3 Aho Corasick . . . 17
8.4 Dynamic Aho Corasick . . . 18
8.5 Suffix Array . . . 18
8.6 Hashing . . . 18
8.7 Palindromic Tree . . . 18
8.8 Manacher . . . 19

9 Dp **19**
9.1 Convex Hull Trick . . . 19
9.2 Knuth Division . . . 19
9.3 Li Chao Tree . . . 19

10 Geometry 2D **19**
10.1 Point . . . 19
10.2 Line . . . 20
10.3 Circle . . . 21
10.4 Tangent . . . 22
10.5 Convex Hull . . . 22
10.6 Polygon . . . 22

11 Miscellaneous **22**
11.1 Rnd template . . . 22

1 Template

```
// :'<,'>w !sed 's/\/\/.*//' | tr -d '[:space:]' | tr A-Z a-z | md5sum | cut -c-6
//echo "vnoremap <space>h :w !sed 's/\/\/.*//' | tr -d '[:space:]' | tr A-Z a-z | md5sum | cut -c-6<CR>" >> ~/.vimrc
#include <bits/stdc++.h>

using namespace std;

#define all(v) v.begin(),v.end()
#define rall(v) v.rbegin(),v.rend()
#define pb push_back
#define sz(a) int(a.size())
#define F first
#define S second
typedef long long ll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef pair<int,int> pii;
typedef pair<ll,ll> pll;
const int MOD = 1e9+7;
const int INF = INT_MAX;
const ll INF64 = LLONG_MAX;
const long double EPS = 1e-9;
const long double PI = acosl(-1.0L);
void setIO(string p){
  freopen((p + ".in").c_str(), "r", stdin);
  freopen((p + ".out").c_str(), "w", stdout);
}

void solve(){

}


int main(){
  ios_base::sync_with_stdio(0);
  cin.tie(0);
  cout.tie(0);
  int tc = 1;
  //cin >> tc;
  for (int t = 1; t <= tc; t++)solve();
}
```

2 Data Structures

2.1 Segment Tree

```
// TOTAL: c88465
struct Node{
  ll val = 0;
}; // #6368df
Node operator+(Node a,Node b){return {a.val+b.val};}
class Segment_Tree{
public:
  int n;
  vector<Node> tree;

  Segment_Tree(int x = 1e5+10){
    n = x;
    tree.resize(2*n+1);
  } // #9ba7b9

  void update(int p, ll val){
    for(tree[p+=n] = {val};p>1;p>>=1)
      tree[p>>1] = tree[p]+tree[p^1];
    //for (tree[p+=n] = {val};p>>= 1;)
    //  t[p] = t[p<<1]+t[p<<1|1]; // #9c6ca8
  } // #fd122a
  ll query(int l,int r){
    Node s;
    for(l+=n,r+=n ; l<r ; l>>=1 , r>>=1){
      if(l&1)s = s+ tree[l++];
      if(r&1)s = s+ tree[--r];
    }
    return s.val;
  } // #409731
}; // #19ff3f
```

2.2 Merge Sort Tree

```
// Hash: #545fdb
struct Node{
  vi val;

  int search(int x){
    // cnt >= x
    auto it = lower_bound(all(val), x);
    return val.end() - it; // #c83330
    // cnt > x
    //auto it = upper_bound(all(val), x);
    //return val.end() - it; // #fe1bbd
    // cnt == x
    //auto l_it = lower_bound(all(val), x);
    //auto u_it = upper_bound(all(val), x);
    //return u_it - l_it; // #281439
  }
};

Node operator+(Node a,Node b){
  Node aux;
  aux.val.resize(sz(a.val) + sz(b.val));
  merge(all(a.val), all(b.val), aux.val.begin());
} // #086024
```

2.3 Segment Tree Recursive

```
//Hash: #f94ebe
struct Node{
  ll val = 0;
};
Node operator+(Node a,Node b){return {a.val+b.val};}
class Segment_Tree{
public:
  int n;
  vector<Node> tree;

  Segment_Tree(int x = 1e5+10){
    n = x;
    tree.resize(4*n+1);
  } // #f71c07

  void update(int v, int tl, int tr, int pos, int new_val){
    if(tl == tr){
      tree[v] = {new_val};
      return;
    }

    int tm = tl+(tr-tl)/2;

    if (pos<=tm)update(v<<1,tl,tm,pos,new_val);
    else update((v<<1)+1,tm+1,tr,pos,new_val);

    tree[v] = tree[v<<1] + tree[(v<<1)+1];
  } // #b8d734

  Node query(int v, int tl, int tr, int l, int r){
    if (l > r)return {0};
    if (l==tl && r==tr)return tree[v];

    int tm = tl+(tr-tl)/2;
    return query(v<<1,tl,tm,l,min(r,tm))+query((v<<1)+1,tm+1,tr,
      max(l,tm+1),r);
  } // #3499b3
};
```

2.4 Lazy

```
// Hash: c02121
struct Segment_Tree{

  int n;
  vector<Node> tree;
  vector<Lazy> lazy; // #b7ee33

  Segment_Tree(int x = 1e5+10){
    n = x;
    tree.resize(4*n);
    lazy.resize(4*n);
  } // #a86d70

  void push_set(int v,int l,int r){
    if(!lazy[v].has_set)return;
    tree[v].val = lazy[v].val*(r-l+1);

    if(l!=r){
      lazy[v<<1] = lazy[v];
      lazy[(v<<1)|1] = lazy[v];
    }
    lazy[v].has_set = 0;
    lazy[v].val = 0;
  } // #fd22cc

  void push_sum(int v,int l,int r){
    if(!lazy[v].has_sum)return;
    tree[v].val = tree[v].val+lazy[v].val*(r-l+1);
    if(l!=r){
      lazy[v<<1].val = lazy[v<<1].val + lazy[v].val;
      lazy[(v<<1)|1].val = lazy[(v<<1)|1].val + lazy[v].val;
      if(!lazy[v<<1].has_set)lazy[v<<1].has_sum=1;
      if(!lazy[(v<<1)|1].has_set)lazy[(v<<1)|1].has_sum=1;
    }
    lazy[v].has_sum = 0;
    lazy[v].val= 0;
  } // #a6dba9

  void update(int pos,ll val,int v,int tl, int tr){
    if(tl == tr)tree[v] = {val};
    else{
      int tm = tl+(tr-tl)/2;
      if(pos <= tm)update(pos,val,v<<1,tl,tm);
      else update(pos,val,(v<<1)+1,tm+1,tr);
      tree[v] = tree[v<<1]+tree[(v<<1)+1];
    }
  } // #19c7ff

  void update_set(int v,int tl,int tr,int l,int r,ll val){
    push_set(v,tl,tr);
    push_sum(v,tl,tr);
    if(l>tr||r<tl)return;
    if(tl>= l && tr <=r){
      lazy[v].has_set=1;
      lazy[v].has_sum=0;
```

```
      lazy[v].val=val;
      push_set(v,tl,tr);
      return;
    }
    int tm = tl+(tr-tl)/2;
    update_set(v<<1,tl,tm,l,r,val);
    update_set((v<<1)|1,tm+1,tr,l,r,val);
    tree[v] = tree[v<<1]+tree[(v<<1)|1];
  } // #91ab68

  void update_sum(int v,int tl,int tr,int l,int r,ll val){
    push_set(v,tl,tr);
    push_sum(v,tl,tr);
    if(l>tr||r<tl)return;
    if(tl>= l && tr <=r){
      lazy[v].has_sum=1;
      lazy[v].val += val;
      push_sum(v,tl,tr);
      return;
    }
    int tm = tl+(tr-tl)/2;
    update_sum(v<<1,tl,tm,l,r,val);
    update_sum((v<<1)|1,tm+1,tr,l,r,val);
    tree[v] = tree[v<<1]+tree[(v<<1)|1];
  } // #e2f028

  Node query(int v ,int tl ,int tr ,int l ,int r){
    push_set(v,tl,tr);
    push_sum(v,tl,tr);
    if(l>tr||r<tl)return {0};
    if(tl>= l && tr <= r )
      return tree[v];
    int tm = tl+(tr-tl)/2;
    return query(v<<1,tl,tm,l,r)+query((v<<1)|1,tm+1,tr,l,r);
  } // #3cd2a5

};
```

2.5 Persistent Segment Tree

```
// Hash: #437d7b
struct Node{
  ll val;
  Node *l,*r;
  Node():val(0),l(nullptr),r(nullptr){}
}; // #736465

ll get(Node* t){
  return t?t->val:0;
} // #dea637

Node *update(Node *t,int tl,int tr,int pos,int v){
  Node* aux = new Node();
  if(tl == tr){
    aux->val = v;
    return aux;
  }

  int tm = tl+(tr-tl)/2;

  Node* l = t?t->l:nullptr;
  Node* r = t?t->r:nullptr;

  if(pos <= tm){
    aux->l = update(l,tl,tm,pos,v);
    aux->r = r;
  }else{
    aux->l = l;
    aux->r = update(r,tm+1,tr,pos,v);
  }

  aux->val = get(aux->l)+get(aux->r);
  return aux;
} // #a8f7ef

ll query(Node *t,int tl,int tr,int l,int r){
  if(!t || l>r)return 0;
  if(l==tl && r == tr)return t->val;
  int tm = tl+(tr-tl)/2;
  return query(t->l,tl,tm,l,min(r,tm))+query(t->r,tm+1,tr,max(l,tm
      +1),r);
} // #4596dc
```

2.6 Dynamic Segment Tree

```
struct Vertex {
int left, right;
int sum = 0;
Vertex *left_child = nullptr, *right_child = nullptr;
Vertex(int lb, int rb) {
left = lb;
right = rb;
}
void extend() {
if (!left_child && left + 1 < right) {
int t = (left + right) / 2;
left_child = new Vertex(left, t);
right_child = new Vertex(t, right);
}
}
void add(int k, int x) {
extend();
sum += x;
if (left_child) {
if (k < left_child->right)
left_child->add(k, x);
else
right_child->add(k, x);
}
}
int get_sum(int lq, int rq) {
if (lq <= left && right <= rq)
return sum;
if (max(left, lq) >= min(right, rq))
return 0;
extend();
return left_child->get_sum(lq, rq) + right_child->get_sum(lq, rq);
}
};
```

2.7 Sparse Table

```
//Hash: #a9e313
struct Node{
  int val = 0;
  Node operator+(Node b){return {__gcd(val,b.val)};}
}; // #f7df3b

class SparseTable{
  public:
  vector<vector<Node>> sparse;

  SparseTable(int n=2e5+10){
    sparse.assign(n , vector<Node> (log2(n)+1));
  } // #031899

  void build(const vi &a){
    for(int i=0;i<sz(a);i++)
      sparse[i][0] = {a[i]};

    for(int j=1;(1<<j)<=sz(a);j++)
      for(int i=0;i+(1<<j)-1 <sz(a);i++)
        sparse[i][j]= sparse[i][j-1]+sparse[i+(1<<(j-1))][j-1];
  } // #139c7b

  int query(int l, int r){
    int len = r-l+1;
    int k = log2(len);
    return (sparse[l][k]+sparse[r-(1<<k)+1][k]).val;
  } // #db37a1
};
```

2.8 Treap

```
struct node {
    int val, sz, prior, lazy, sum, mx, mn, repl;
    bool repl_flag, rev;
    node *l, *r, *par;
    node() {
      lazy = 0;
      rev = 0;
      sum = 0;
      val = 0;
      sz = 0;
      mx = 0;
      mn = 0;
      repl = 0;
      repl_flag = 0;
      prior = 0;
      l = NULL;
      r = NULL;
      par = NULL;
    }
    node(int _val) {
      val = _val;
      sum = _val;
      mx = _val;
      mn = _val;
      repl = 0;
      repl_flag = 0;
      rev = 0;
      lazy = 0;
      sz = 1;
      prior = rnd();
      l = NULL;
      r = NULL;
      par = NULL;
    }
  };

struct treap {
  typedef node* pnode;
  pnode root;
  map<int, pnode> position;//positions of all the values
  //clearing the treap
  void clear() {
    root = NULL;
    position.clear();
  }

  treap() {
    clear();
  }

```

```
int size(pnode t) {
  return t ? t->sz : 0;
}
void update_size(pnode &t) {
  if(t) t->sz = size(t->l) + size(t->r) + 1;
}

void update_parent(pnode &t) {
  if(!t) return;
  if(t->l) t->l->par = t;
  if(t->r) t->r->par = t;
}
//add operation
void lazy_sum_upd(pnode &t) {
  if( !t or !t->lazy )    return;
  t->sum += t->lazy * size(t);
  t->val += t->lazy;
  t->mx += t->lazy;
  t->mn += t->lazy;
  if( t->l )  t->l->lazy += t->lazy;
  if( t->r )  t->r->lazy += t->lazy;
  t->lazy = 0;
}
//replace update
void lazy_repl_upd(pnode &t) {
  if( !t or !t->repl_flag )   return;
  t->val = t->mx = t->mn = t->repl;
  t->sum = t->val * size(t);
  if( t->l ) {
    t->l->repl = t->repl;
    t->l->repl_flag = true;
  }
  if( t->r ) {
    t->r->repl = t->repl;
    t->r->repl_flag = true;
  }
  t->repl_flag = false;
  t->repl = 0;
}
//reverse update
void lazy_rev_upd(pnode &t) {
  if( !t or !t->rev ) return;
  t->rev = false;
  swap(t->l, t->r);
  if( t->l )  t->l->rev ^= true;
  if( t->r )  t->r->rev ^= true;
}
//reset the value of current node assuming it now
//represents a single element of the array
void reset(pnode &t) {
  if(!t) return;
  t->sum = t->val;
  t->mx = t->val;
  t->mn = t->val;
}
//combine node l and r to form t by updating corresponding
   queries
void combine(pnode &t, pnode l, pnode r) {
  if(!l) {
    t = r;
    return;
  }
  if(!r) {
    t = l;
    return;
  }
  //Beware!!!Here t can be equal to l or r anytime
  //i.e. t and (l or r) is representing same node
  //so operation is needed to be done carefully
  //e.g. if t and r are same then after t->sum=l->sum+r->sum
    operation,
  //r->sum will be the same as t->sum
  //so BE CAREFUL
  t->sum = l->sum + r->sum;
  t->mx = max(l->mx, r->mx);
  t->mn = min(l->mn, r->mn);
}
//perform all operations
void operation(pnode &t) {
  if( !t )    return;
  reset(t);
  lazy_rev_upd(t->l);
  lazy_rev_upd(t->r);
  lazy_repl_upd(t->l);
  lazy_repl_upd(t->r);
  lazy_sum_upd(t->l);
  lazy_sum_upd(t->r);
  combine(t, t->l, t);
  combine(t, t, t->r);
}
//split node t in l and r by key k
//so first k+1 elements(0,1,2,...k) of the array from node t
//will be split in left node and rest will be in right node
void split(pnode t, pnode &l, pnode &r, int k, int add = 0) {
  if(t == null) {
    l = null;
    r = null;
    return;
  }
  lazy_rev_upd(t);
  lazy_repl_upd(t);
  lazy_sum_upd(t);
  int idx = add + size(t->l);
  if(t->l) t->l->par = null;
  if(t->r) t->r->par = null;
  if(idx <= k)
    split(t->r, t->r, r, k, idx + 1), l = t;
  else
    split(t->l, l, t->l, k, add), r = t;

  update_parent(t);
  update_size(t);
  operation(t);
}
//merge node l with r in t
void merge(pnode &t, pnode l, pnode r) {
  lazy_rev_upd(l);
  lazy_rev_upd(r);
  lazy_repl_upd(l);
  lazy_repl_upd(r);
  lazy_sum_upd(l);
  lazy_sum_upd(r);
  if(!l) {
    t = r;
    return;
  }
  if(!r) {
    t = l;
    return;
  }

  if(l->prior > r->prior)
    merge(l->r, l->r, r), t = l;
  else
    merge(r->l, l, r->l), t = r;

  update_parent(t);
  update_size(t);
  operation(t);
}
//insert val in position a[pos]
//so all previous values from pos to last will be right shifted
void insert(int pos, int val) {
  if(root == NULL) {
    pnode to_add = new node(val);
    root = to_add;
    position[val] = root;
    return;
  }

  pnode l, r, mid;
  mid = new node(val);
  position[val] = mid;

  split(root, l, r, pos - 1);
  merge(l, l, mid);
  merge(root, l, r);

/*pnode x = new Node(val),merge(root,root,x);*/
}
//erase from qL to qR indexes
//so all previous indexes from qR+1 to last will be left shifted
    qR-qL+1 times
void erase(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  merge(root, l, r);
}
//returns answer for corresponding types of query
int query(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);

  int answer = mid->sum;
  merge(r, mid, r);
  merge(root, l, r);

  return answer;
}
//add val in all the values from a[qL] to a[qR] positions
void update(int qL, int qR, int val) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
  lazy_repl_upd(mid);
  mid->lazy += val;

  merge(r, mid, r);
  merge(root, l, r);
}
//reverse all the values from qL to qR
void reverse(int qL, int qR) {
  pnode l, r, mid;

  split(root, l, r, qL - 1);
  split(r, mid, r, qR - qL);
```

```
249
250   if(mid)mid->rev ^= 1;
251   merge(r, mid, r);
252   merge(root, l, r);
253 }
254 //replace all the values from a[qL] to a[qR] by v
255 void replace(int qL, int qR, int v) {
256   pnode l, r, mid;
257
258   split(root, l, r, qL - 1);
259   split(r, mid, r, qR - qL);
260   lazy_sum_upd(mid);
261   mid->repl_flag = 1;
262   mid->repl = v;
263   merge(r, mid, r);
264   merge(root, l, r);
265 }
266 //it will cyclic right shift the array k times
267 //so for k=1, a[qL]=a[qR] and all positions from ql+1 to qR will
268 //have values from previous a[qL] to a[qR-1]
269 //if you make left_shift=1 then it will to the opposite
270 void cyclic_shift(int qL, int qR, int k, bool left_shift = 0) {
271   if(qL == qR) return;
272   k %= (qR - qL + 1);
273
274   pnode l, r, mid, fh, sh;
275   split(root, l, r, qL - 1);
276   split(r, mid, r, qR - qL);
277
278   if(left_shift == 0) split(mid, fh, sh, (qR - qL + 1) - k - 1);
279   else split(mid, fh, sh, k - 1);
280   merge(mid, sh, fh);
281
282   merge(r, mid, r);
283   merge(root, l, r);
284 }
285 bool exist;
286 //returns index of node curr
287 int get_pos(pnode curr, pnode son = nullptr) {
288   if(exist == 0) return 0;
289   if(curr == NULL) {
290     exist = 0;
291     return 0;
292   }
293   if(!son) {
294     if(curr == root) return size(curr->l);
295     else return size(curr->l) + get_pos(curr->par, curr);
296   }
297
298   if(curr == root) {
299     if(son == curr->l) return 0;
300     else return size(curr->l) + 1;
301   }
302
303   if(curr->l == son) return get_pos(curr->par, curr);
304   else return get_pos(curr->par, curr) + size(curr->l) + 1;
305 }
306 //returns index of the value
307 //if the value has multiple positions then it will
308 //return the last index where it was added last time
309 //returns -1 if it doesn't exist in the array
310 int get_pos(int value) {
311   if(position.find(value) == position.end()) return -1;
312   exist = 1;
313   int x = get_pos(position[value]);
314   if(exist == 0) return -1;
315   else return x;
316   }
317   //returns value of index pos
318   int get_val(int pos) {
319     return query(pos, pos);
320   }
321   //returns size of the treap
322   int size() {
323     return size(root);
324   }
325   //inorder traversal to get indexes chronologically
326   void inorder(pnode cur) {
327     if(cur == NULL) return;
328     operation(cur);
329     inorder(cur->l);
330     cout << cur->val << ' ';
331     inorder(cur->r);
332   }
333
334   bool find(int val) {
335     if(get_pos(val) == -1) return 0;
336     else return 1;
337   }
338 };
339
340 treap t;
341 //Beware!!!here treap is 0-indexed
342
343 int main() {
344   int i, j, k, n, m, l, r, q;
345   for(i = 0; i < 10; i++) t.insert(i, i * 10);
346   t.cyclic_shift(4, 5, 1);
347   t.update(2, 5, 1);
348   t.replace(2, 5, 100);
349   t.reverse(2, 9);
350   t.replace(2, 5, 200);
351   cout << t.query(0, 7) << nl;
352   t.cyclic_shift(2, 3, 2, 1);
353   cout << t.get_pos(20) << nl;
354   t.erase(2, 2);
355   cout << t.find(30) << nl;
356   t.print_array();
357   return 0;
358 }
```

2.9 BIT

```
1 // Hash: b34e73
2 struct BIT{
3   vll bit;
4   int n;
5   BIT(int x){n = x+1;bit.assign(n,0);} // #69335b
6
7   ll get(int x){
8     ll ans =0;
9     for(;x;x-=x&-x)ans+=bit[x];
10    return ans;
11  } // #781a0f
12
13  void update(int x, ll val){
14    for(;x<n;x+=x&-x)
15      bit[x] += val;
16  } // #4e7832
17 };
```

2.10 DSU

```
1 // Hash: 60e0d3
2 class DSU{
3   private:
4   vi parent,rank,size;
5   int c;
6   public:
7   int mx = 0; // #9e8891
8   DSU(int n):parent(n+1),rank(n+1,0),size(n+1,1),c(n){
9     iota(all(parent),0);
10  } // #9dff24
11
12  int find(int u){return parent[u] == u?u:parent[u] = find(parent[
      u]) ;} // #a9e220
13
14  bool same(int u,int v){return find(u)==find(v);} // #df0aa8
15
16  int get_size(int u){return size[find(u)];} // #f7b590
17
18  int count(){return c;} // #466020
19
20  void merge(int u,int v){
21    u = find(u);
22    v = find(v);
23    if(u!=v){
24      c--;
25      if(rank[u] > rank[v])swap(u,v);
26      parent[u] = v;
27      size[v] += size[u];
28      if(rank[u] == rank[v])rank[v]++;
29      mx = max(mx,size[v]);
30    }
31  } // #ee7199
32 };
```

2.11 DSU with Rollback

```
1 class DSU{
2   private:
3   vi parent,rank,size;
4   int c;
5   struct State{
6     int u,v;
7     int rank_v,size_v,mx;
8   };
9   vector<State> history;
10
11  public:
12  int mx = 0;
13  DSU(int n):parent(n+1),rank(n+1,0),size(n+1,1),c(n){
14    iota(all(parent),0);
15  }
16
17  int find(int u){
18    while(u!=parent[u])u = parent[u];
19    return u;
20  }
21
22  bool same(int u,int v){return find(u)==find(v);} // #df0aa8
23
24  int get_size(int u){return size[find(u)];} // #f7b590
25
26  int count(){return c;} // #466020
27
28  int get_time(){return history.size();}
29
30  void rollback(int t){
31    while(history.size()>t){
32      State st = history.back();
33      history.pop_back();
34      int u = st.u;
```

```
      int v = st.v;
      parent[u] = u;
      rank[v] = st.rank_v;
      size[v] = st.size_v;
      mx = st.mx;
      c++;
    }
  }

  void rollback(){
    if(history.empty())return;
    State st = history.back();
    history.pop_back();
    int u = st.u;
    int v = st.v;
    parent[u] = u;
    rank[v] = st.rank_v;
    size[v] = st.size_v;
    mx = st.mx;
    c++;
  }

  void merge(int u,int v){
    u = find(u);
    v = find(v);
    if(u!=v){
      c--;
      if(rank[u] > rank[v])swap(u,v);
      history.pb({u,v,rank[v],size[v],mx});
      parent[u] = v;
      size[v] += size[u];
      if(rank[u] == rank[v])rank[v]++;
      mx = max(mx,size[v]);
    }
  }
};
```

2.12 Dynamic Connectivity

```
struct Dynaco{
  int n;
  vector<vector<pii>> st;
  DSU dsu;//with rollbacks

  Dynaco(int T,int v):n(T),dsu(v){st.resize(4*n+1);}

  void add(int v, int tl, int tr, int l, int r, pii e){
    if(l>r)return;

    if(l==tl&&r==tr){
      st[v].push_back(e);
      return;
    }
    int tm = tl+(tr-tl)/2;
    add(v<<1,tl,tm,l,min(r,tm),e);
    add((v<<1)+1,tm+1,tr,max(l,tm+1),r,e);
  }


  void dfs(int v, int tl, int tr, vi& ans){
    int merges = 0;

    for(auto [x,y]:st[v])
      if(dsu.merge(x,y))merges++;

    if(tl==tr){
      ans[tl] += dsu.count();
    }else{
      int tm = tl+(tr-tl)/2;
      dfs(v<<1,tl,tm,ans);
      dfs((v<<1)+1,tm+1,tr,ans);
    }

    while(merges--)dsu.rollback();
  }
};
```

2.13 Trie

```
struct Node{
  int v;
  int cnt = 0;
  Node* nxt[2];
  Node(int __v){
    v = __v;
    cnt = 0;
    nxt[0] = nxt[1] = NULL;
  }
};

struct Trie{
  typedef Node* pnode;
  pnode root;
  const int B = 30;

  void insert(int val){
    pnode cur = root;
    for(int i=B;i>=0;i--){
      bool b = val>>i&1;
      if(cur->nxt[b]==NULL)cur->nxt[b] = new Node(b);
      cur = cur->nxt[b];
      cur->cnt++;
    }
  }

  pnode erase(pnode cur, int val, int i) {
    if (cur == NULL)return NULL;
    cur->cnt--;
    if (i < 0) {
      if (cur->cnt == 0) {
        delete cur;
        return NULL;
      }
      return cur;
    }

    bool b = (val >> i) & 1;
    cur->nxt[b] = erase(cur->nxt[b], val, i - 1);

    if (cur->cnt == 0) {
      delete cur;
      return NULL;
    }

    return cur;
  }

  int query(int val){
    pnode cur = root;
    int ans = 0;
    for(int i = B;i>=0;i--){
      bool b = val>>i&1;
      if(cur->nxt[!b])cur = cur->nxt[!b],ans<<=1,ans++;
      else cur = cur->nxt[b],ans<<=1;
    }
    return ans;
  }
};
```

2.14 Mo's Algorithm

```
void remove(idx);  // TODO: remove value at idx from data
      structure
void add(idx);     // TODO: add value at idx from data structure
int get_answer();  // TODO: extract the current answer of the data
      structure
 // #86ec16
int block_size;

struct Query{
  int l, r, idx;
  bool operator<(Query other) const
  {
    return make_pair(l / block_size, r) <
      make_pair(other.l / block_size, other.r);
  }
}; // #8e89a9

//in main

sort(all(queries));

// TODO: initialize data structure

int cur_l = 0;
int cur_r = -1;
// invariant: data structure will always reflect the range [cur_l,
      cur_r]
for(Query q:queries){
  while(cur_l > q.l)
    add(--cur_l);

  while(cur_r < q.r)
    add(++cur_r);

  while(cur_l < q.l)
    remove(cur_l++);

  while(cur_r > q.r)
    remove(cur_r--);

  answers[q.idx] = get_answer();
} // #50d13b
```

3 Graphs

3.1 2-SAT

```
class SAT{
private:
  int n;
  vector<vi> adj;
  vector<vi> adj_t;
  vector<vi> adj_cond;
  vi order;
  vector<bool> vis;
  vi who;
  vector<bool> ans;
  int comp;
public:

  SAT(int m=0){
    n = (m<<1);
    adj.resize(n);
```

```
  adj_t.resize(n);
  vis.assign(n,0);
  who.assign(n,-1);
}

void dfs1(int u) {
  vis[u] = true;
  for (int v:adj[u])
    if (!vis[v])dfs1(v);
  order.pb(u);
}

void dfs2(int u) {
  vis[u] = 1;
  for (int v : adj_t[u])
    if (!vis[v])dfs2(v);
  who[u] = comp;
}

//~a->b ~b->a
void addOR(int a, bool af, int b, bool bf) {
  a += a + (af ^ 1);
  b += b + (bf ^ 1);
  adj[a ^ 1].push_back(b);
  adj[b ^ 1].push_back(a);
  adj_t[b].push_back(a ^ 1);
  adj_t[a].push_back(b ^ 1);
}

//( a OR b ) ^ (~a or ~b)
void addXOR(int a, bool af, int b, bool bf) {
  addOR(a, af, b, bf);
  addOR(a, !af, b, !bf);
}
//(a -> b)
void _add(int a,bool af,int b,bool bf) {
  a += a + (af ^ 1);
  b += b + (bf ^ 1);
  adj[a].push_back(b);
  adj_t[b].push_back(a);
}

//(a->b)^(b->a)
void add(int a,bool af,int b,bool bf) {
  _add(a, af, b, bf);
  _add(b, !bf, a, !af);
}

bool solve_SAT(){
  for(int i=0;i<n;i++)
    if(!vis[i])dfs1(i);

  comp = -1;
  vis.assign(n,0);
  for(int i=sz(order)-1;i>=0;i--)
    if(!vis[order[i]]){
      comp++;
      dfs2(order[i]);
    }
  ans.assign(n/2,0);
  for(int i=0;i<n;i+=2){
    if(who[i] == who[i+1])return 0;
    ans[i/2] = (who[i] > who[i+1]);
  }
  return 1;
}

```

```
  bool operator[](int i){
    return ans[i];
  }

};
```

3.2 Block Cut Tree

```
vector<vi> adj(n+1),bcc(n+1);
vi disc(n+1),low(n+1),who(n+1);
stack<int> st;
int Time = 0,sz = 0,x;

auto dfs = [&](auto &self,int u,int p)->void{
  disc[u] = low[u] = ++Time;
  st.push(u);
  for(int v:adj[u]){
    if(!disc[v]){
      self(self,v,u);
      low[u] = min(low[u],low[v]);
      if(low[v] >= disc[u]){
        sz++;
        do{
          x = st.top();
          st.pop();
          bcc[x].pb(sz);
        }while(x^v);
        bcc[u].pb(sz);
      }
    }else if(v!=p)low[u] = min(low[u],disc[v]);
  }
};

for(int i=1;i<=n;i++)
  if(!disc[i])dfs(dfs,i,0);

int c = sz+n+2;
vector<vi> tree(c);
vector<bool> art(c,0);
for(int i=1;i<=n;i++){
  if(sz(bcc[i])>1){
    who[i] = ++sz;
    art[who[i]] = 1;
    for(int v:bcc[i]){
      tree[who[i]].pb(v);
      tree[v].pb(who[i]);
    }
  }else if(sz(bcc[i])==1)who[i] = bcc[i][0];
}
```

3.3 Bellman-Ford Cycle

```
vector<int> d(n, INF);
d[v] = 0;
vector<int> p(n, -1);
int x;
for (int i = 0; i < n; ++i) {
x = -1;
for (Edge e : edges)
    if (d[e.a] < INF)
  if (d[e.b] > d[e.a] + e.cost) {
      d[e.b] = max(-INF, d[e.a] + e.cost);
      p[e.b] = e.a;
      x = e.b;
  }
}
if (x == -1)cout << "No negative cycle from " << v;
else{
int y = x;
for (int i = 0; i < n; ++i)y = p[y];
vector<int> path;
for (int cur = y;; cur = p[cur]) {
    path.push_back(cur);
    if (cur == y && path.size() > 1)break;
}
reverse(path.begin(), path.end());
}
```

3.4 Bellman-Ford Path

```
vector<int> d(n, INF);
d[v] = 0;
vector<int> p(n, -1);

for (;;) {
bool any = false;
for (Edge e : edges)
    if (d[e.a] < INF)
  if (d[e.b] > d[e.a] + e.cost) {
      d[e.b] = d[e.a] + e.cost;
      p[e.b] = e.a;
      any = true;
  }
if (!any)
    break;
}

if (d[t] == INF)
  cout << "No path from " << v << " to " << t << ".";
else {
  vector<int> path;
  for (int cur = t; cur != -1; cur = p[cur])path.push_back(cur);
  reverse(path.begin(), path.end());
}
```

3.5 Bipartite Check

```
vector<vi> color;
bool ok = 1;
queue<int> q;
for(int i=0;i<n;i++){
  if(a[i]!=0)continue;
  q.push(i);
  color[i] = 1;
  while (!q.empty()) {
    int v = q.front();
    q.pop();
    for(int u:adj[v]){
      if(a[u]==0){
        a[u] = a[v]^3;
        q.push(u);
      }else{
        valid &= a[u]!=a[v];
      }
    }
  }
}
```

3.6 Bridges

```

vector<pii> bridge;
vi disc(MAX),low(MAX);
vector<vi> adj(MAX);
int Time = 0;

void dfs(int u,int p){
```

```
  disc[u] = low[u] = ++Time;
  for(int v:adj[u]){
    if(v==p)continue;
    if(!disc[v]){
      dfs(v,u);
      // <= for articulation points
      if(disc[u]<low[v])bridge.emplace_back(u,v);
      low[u] = min(low[u],low[v]);
    }else{
      low[u] = min(low[u],disc[v]);
    }

  }
}
```

3.7 Bridge Tree

```
//Bridges alredy found
vi who(n+1,-1);
int c = 1;

auto dfs2 = [&](auto &self,int u)->void{
  who[u] = c;
  for(auto [v,i]:adj[u]){
    if(who[v]!=-1||bridge[i])continue;
    self(self,v);
  }
};


for(int i=1;i<=n;i++){
  if(who[i]==-1)dfs2(dfs2,i),c++;
}

vector<vi> tree(c);

for(int i=0;i<m;i++){
  if(bridge[i]){
    tree[who[edges[i].u]].pb(who[edges[i].v]);
    tree[who[edges[i].v]].pb(who[edges[i].u]);
  }
}
```

3.8 Dijkstra

```
vll dist(n,INF64);
vector<bool> vis(n,0);
priority_queue<pll,vector<pll>,greater<pll>> pq;
pq.emplace(0,0);
dist[0] = 0;
vll cnt(n,0);//number of shortest paths
while(!pq.empty()){
  auto [d1,u] = pq.top();
  pq.pop();
  if(vis[u])continue;
  vis[u] = 1;
  for(auto [v,c]:adj[u]){
    if(dist[u]+c < dist[v]){
      dist[v] = dist[u]+c;
      pq.emplace(dist[v],v);
      cnt[v] = cnt[u];
    }else if(dist[u]+c == dist[v])(cnt[v]+=cnt[u])%=MOD;
  }
}
```

3.9 Dinic

```
struct flowEdge{
int u,v;
ll cap,flow = 0;
flowEdge(int u, int v, ll cap) : u(u), v(v), cap(cap) {};
};

struct Dinic{
vector<flowEdge> edges;
vector<vi> adj;
int n,s,t;
int id = 0;
vi level, next;

queue<int> q;

Dinic(int n, int s, int t) : n(n), s(s), t(t) {
  adj.resize(n);
  level.resize(n);
  next.resize(n);
  fill(all(level),-1);
  level[s] = 0;
  q.push(s);
}

void add(int u, int v, ll cap){
  edges.emplace_back(u,v,cap);
  edges.emplace_back(v,u,0);
  adj[u].pb(id++);
  adj[v].pb(id++);
}

bool bfs(){
  while (!q.empty()){
    int curr = q.front();
    q.pop();
    for (auto e : adj[curr]){
      if (edges[e].cap - edges[e].flow  < 1) continue;
      if (level[edges[e].v] != -1) continue;
      level[edges[e].v] = level[edges[e].u] + 1;
      q.push(edges[e].v);
    }
  }
  return level[t] != -1;
}

ll dfs(int u, ll flow){
  if (flow == 0) return 0;
  if (u == t) return flow;
  for (auto& cid = next[u]; cid < adj[u].size(); cid++){
      int e = adj[u][cid];
      int v = edges[e].v;

      if (level[edges[e].u] + 1 != level[v] || edges[e].cap -
     edges[e].flow < 1) continue;
      ll f = dfs(v,min(flow,edges[e].cap - edges[e].flow));
      if (f == 0) continue;
      edges[e].flow += f;
      edges[e ^ 1].flow -= f;
      return f;
  }
  return 0;
}

ll maxFlow(){
  ll flow = 0;
  while (bfs()){
      fill(all(next),0);
      for (ll f = dfs(s,INF64); f != 0ll; f = dfs(s,INF64)) flow
     += f;
      fill(all(level),-1);
      level[s] = 0;
      q.push(s);
  }
  return flow;
    }


vector<pii> minCut(){
  vector<pii> ans;
  fill(all(level),-1);
  level[s] = 0;
  q.push(s);
  while (!q.empty()){
    int curr = q.front();
    q.pop();
    for (int id = 0; id < adj[curr].size(); id++){
      int e = adj[curr][id];
      if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow
     > 0){
        q.push(edges[e].v);
        level[edges[e].v] = level[edges[e].u] + 1;
      }
    }
  }

  for (int i = 0; i < level.size(); i++){
    if (level[i] != - 1){
      for (int id = 0; id < adj[i].size(); id++){
      int e = adj[i][id];
      if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow
     == 0)
        ans.emplace_back(edges[e].u,edges[e].v);
      }
    }
  }
  return ans;
}

vector<pii> MaximumMatching(){
  vector<pii> ans;
  fill(all(level),-1);
  level[s] = 0;
  q.push(s);
  while (!q.empty()){
    int curr = q.front();
    q.pop();
    for (int id = 0; id < adj[curr].size(); id++){
    int e = adj[curr][id];
    if (level[edges[e].v] == -1 && edges[e].cap - edges[e].flow ==
      0 && edges[e].flow != 0ll){
        q.push(edges[e].v);
        level[edges[e].v] = level[edges[e].u] + 1;
      }
    }
  }
  for (int i = 0; i < level.size(); i++){
    if (level[i] != - 1)
      for (int id = 0; id < adj[i].size(); id++){
        int e = adj[i][id];
        if (edges[e].u != s && edges[e].v != t && edges[e].cap -
      edges[e].flow == 0 && edges[e].flow != 0ll)
          ans.emplace_back(edges[e].u,edges[e].v);
      }
  }
  return ans;
}
```

```
};
```

3.10 Euler Path Directed

```
struct EulerSolver{
private:
  vector<vector<pii>> adj;
  int n,edges;
  vi in,out;
  vi ans,edge_ans;
  int root;
  vi done;
  void dfs(int u){
    while(done[u] < sz(adj[u])){
      auto [v,id] = adj[u][done[u]++];
      dfs(v);
      edge_ans.pb(id);
    }
    ans.pb(u);
  }
public:
  EulerSolver(vector<vector<pii>> &a,int t):adj(a){
    n = t;
    edges = 0;
    in.assign(n,0);
    out.assign(n,0);
    done.assign(n,0);
    root = -1;
    for(int u=0;u<n;u++)
      for(auto v:adj[u]){
        edges++;
        in[v.F]++;
        out[u]++;
      }

  }

  bool possible(){
    int cnt1 =0,cnt2 =0;
    bool ok = 1;
    for(int i=0;i<n;i++){
      if (in[i] - out[i] == 1) cnt1++;
      if (out[i] - in[i] == 1) cnt2++, root = i;
      if (abs(in[i] - out[i]) > 1) ok = 0;
    }
    if(cnt1 > 1 || cnt2 > 1)ok = 0;
    if(!ok)return 0;

    if(root == -1)
      for(int i=0;i<n;i++)
        if(out[i])root = i;

    if(root == -1)return 1;
    dfs(root);

    if(sz(ans) != edges+1)return 0;
    reverse(all(ans));
    reverse(all(edge_ans));
    return 1;
  }

  int size(){
    return ans.size();
  }

  int operator()(int i,bool x){
    if(x)return ans[i];
    else return edge_ans[i];
  }
};
```

3.11 Euler Path Undirected

```
struct EulerSolver{
private:
  vector<vector<pii>> adj;
  int n,edges;
  vi ans,edge_ans;
  vi deg;
  int root;
  vi done;
  vector<bool> vis;

  void dfs(int u){
    while(done[u] < sz(adj[u])){
      auto [v,id] = adj[u][done[u]++];
      if(vis[id])continue;
      vis[id]=1;
      dfs(v);
      edge_ans.pb(id);
    }
    ans.pb(u);
  }
public:
  EulerSolver(vector<vector<pii>> &a,int t):adj(a),n(t){
    edges = 0;
    done.assign(n,0);
    deg.assign(n,0);
    root = -1;
    for(int u=0;u<n;u++)
      for(auto v:adj[u]){
        edges++;
        deg[v.F]++;
        deg[u]++;
      }
    vis.assign(edges/2,0);
  }

  bool possible(){
    int odd = 0;
    for(int i=0;i<n;i++)
      if((deg[i]/2)&1){
        odd++;
        root = i;
      }


    if(odd>2)return 0;

    if(root == -1)
      for(int i=0;i<n;i++)
        if(deg[i]){root = i;break;}

    if(root == -1)return 1;
    dfs(root);

    if(sz(ans) != edges/2+1)return 0;
    reverse(all(ans));
    reverse(all(edge_ans));
    return 1;
  }

  int size(){
    return ans.size();
  }

  int operator()(int i,bool x){
    if(x)return ans[i];
    else return edge_ans[i];
  }
};
```

3.12 Flood Fill

```
int dx[] = {1,1,0,-1,-1,-1,0,1};
int dx[] = {0,1,1,1,0,-1,-1,-1};

int floodfill(int x,int y,char color1,char color2){
  if(grid[x][y] == "outside")return 0;
  if(grid[x][y] != color1)return 0;
  int ans = 1;
  grid[x][y] = color2;
  for(int d=0;d<8;d++)ans += floodfill(x+dx[d],y+dy[d],color1,
     colr2);
  return ans;
}
```

3.13 Floyd-Warshall

```
for (int i = 0; i < n; ++i)d[i][i] = 0;

for (int k = 0; k < n; ++k)
    for (int i = 0; i < n; ++i)
        for (int j = 0; j < n; ++j)
            if (d[i][k] < INF && d[k][j] < INF)
                d[i][j] = min(d[i][j], d[i][k] + d[k][j]);
```

3.14 Kruskal

```
struct Edge{
  int u,v,w;
  bool operator < (Edge b){return w<b.w;}
};

//inside main

vector<Edge> edges;
sort(all(edges));

DSU dsu(vertices);

for(int i=0;i<E;i++){
  Edge front = edges[i];
  if(!dsu.same(front.u,front.v)){
    cost+= front.w;
    dsu.merge(front.u,front.v);
  }
}
```

3.15 Maximum Clique

```
typedef vector<bitset<200>> vb;
struct Maxclique{
  double limit=0.025, pk=0;
  struct Vertex { int i, d=0; };
  typedef vector<Vertex> vv;
  vb e;
  vv V;
  vector<vi> C;
  vi qmax, q, S, old;
  void init(vv& r) {
    for (auto& v : r) v.d = 0;
    for (auto& v : r) for (auto j : r) v.d += e[v.i][j.i];
    sort(all(r), [](auto a, auto b) { return a.d > b.d; });
```

```
    int mxD = r[0].d;
    for(int i=0;i<sz(r);i++) r[i].d = min(i, mxD) + 1;
  }
  void expand(vv& R, int lev = 1) {
    S[lev] += S[lev - 1] - old[lev];
    old[lev] = S[lev - 1];
    while (sz(R)) {
      if (sz(q) + R.back().d <= sz(qmax)) return;
      q.push_back(R.back().i);
      vv T;
      for(auto v:R) if (e[R.back().i][v.i]) T.push_back({v.i});
      if (sz(T)) {
        if (S[lev]++ / ++pk < limit) init(T);
        int j = 0, mxk = 1, mnk = max(sz(qmax) - sz(q) + 1, 1);
        C[1].clear(), C[2].clear();
        for (auto v : T) {
          int k = 1;
          auto f = [&](int i) { return e[v.i][i]; };
          while (any_of(all(C[k]), f)) k++;
          if (k > mxk) mxk = k, C[mxk + 1].clear();
          if (k < mnk) T[j++].i = v.i;
          C[k].push_back(v.i);
        }
        if (j > 0) T[j - 1].d = 0;
        for(int k=mnk;k<mxk + 1;k++) for (int i : C[k])
          T[j].i = i, T[j++].d = k;
        expand(T, lev + 1);
      } else if (sz(q) > sz(qmax)) qmax = q;
      q.pop_back(), R.pop_back();
    }
  }
  vi maxClique() { init(V), expand(V); return qmax; }
  Maxclique(vb conn) : e(conn), C(sz(e)+1), S(sz(C)), old(S) {
    for(int i=0;i<sz(e);i++)V.push_back({i});
  }
};
```

3.16 MCMF

```
struct MCMF{
  struct edge{
    int u, v;
    ll cap, cost;
    int id;
    edge(int _u, int _v, ll _cap, ll _cost, int _id) {
      u = _u;
      v = _v;
      cap = _cap;
      cost = _cost;
      id = _id;
    }
  };

  int n, s, t, mxid;
  ll flow, cost;
  vector<vi> g;
  vector<edge> e;
  vll d, potential, flow_through;
  vi par;
  bool neg;

  MCMF(){}
  //0 indexed
  MCMF(int _n){
    n = _n + 10;
    g.assign(n, vector<int> ());
    neg = false;
    mxid = 0;
  }

  void add_edge(int u, int v, ll cap, ll cost, int id = -1, bool
     directed = true) {
    if(cost < 0) neg = true;
    g[u].push_back(e.size());
    e.push_back(edge(u, v, cap, cost, id));
    g[v].push_back(e.size());
    e.push_back(edge(v, u, 0, -cost, -1));
    mxid = max(mxid, id);
    if(!directed) add_edge(v, u, cap, cost, -1, true);
  }

  bool dijkstra(){
    par.assign(n, -1);
    d.assign(n, INF64);
    priority_queue <pll,vector<pll>,greater<pll>> q;
    d[s] = 0;
    q.push({0, s});

    while (!q.empty()){
      auto [nw,u] = q.top();
      q.pop();
      if(nw != d[u])continue;
      for (int i = 0; i < sz(g[u]); i++) {
        int id = g[u][i];
        int v = e[id].v;
        ll cap = e[id].cap;
        ll w = e[id].cost + potential[u] - potential[v];
        if (d[u]+w < d[v] && cap > 0) {
          d[v] = d[u] + w;
          par[v] = id;
          q.push({d[v], v});
        }
      }
    }

    for (int i = 0; i < n; i++){
      if (d[i] < INF64)
        d[i] += (potential[i] - potential[s]);
    }

    for (int i = 0; i < n; i++){
      if (d[i] < INF64) potential[i] = d[i];
    }
    return d[t] != INF64; // for max flow min cost
    // return d[t] <= 0; // for min cost flow
  }

  ll send_flow(int v, ll cur){
    if(par[v] == -1) return cur;
    int id = par[v];
    int u = e[id].u;
    ll w = e[id].cost;
    ll f = send_flow(u, min(cur, e[id].cap));
    cost += f * w;
    e[id].cap -= f;
    e[id ^ 1].cap += f;
    return f;
  }

  pll solve(int _s, int _t, ll goal = INF64) {
    s = _s;
    t = _t;
    flow = 0, cost = 0;
    potential.assign(n, 0);
    if(neg){
      d.assign(n, INF);
      d[s] = 0;
      bool relax = true;
      for (int i = 0; i < n && relax; i++) {
        relax = false;
        for (int u = 0; u < n; u++) {
          for (int k = 0; k < (int)g[u].size(); k++) {
            int id = g[u][k];
            int v = e[id].v;
            ll cap = e[id].cap, w = e[id].cost;
            if (d[v] > d[u] + w && cap > 0) {
              d[v] = d[u] + w;
              relax = true;
            }
          }
        }
      }
      for(int i = 0; i < n; i++) if(d[i] < INF) potential[i] = d[i
      ];
    }
    while (flow < goal && dijkstra()) flow += send_flow(t, goal -
     flow);
    flow_through.assign(mxid + 10, 0);
    for (int u = 0; u < n; u++){
      for (auto v : g[u])
        if (e[v].id >= 0) flow_through[e[v].id] = e[v ^ 1].cap;
    }

    return make_pair(flow, cost);
  }
};
```

3.17 Prims

```
vector<bool> vis(n);
priority_queue<pii, vector<pii>, greater<pii>> q;

auto process = [&](int u){
  vis[u] = 1;
  for(int i=0;i<sz(adj[u]);i++){
    pii v = adj[u][i];
    swap(v.S,v.F);
    q.push(v);
  }
}
process(0);

while(!q.empty()){
  auto [w,u] = q.top();q.pop();
  if(!vis[u]){cost += w;process(u);}
}
```

3.18 SCC Kosaraju

```
class SCC{
private:
  int n;
  vector<vi> adj;
  vector<vi> adj_t;
  vector<vi> adj_cond;
  vi order;
  vector<bool> vis;
  vi who;
  int comp;
public:
  SCC(int n,vector<vi> &a,vector<vi>&b):n(n),adj(a),adj_t(b){
    comp = -1;
    who.resize(n);
```

```
    vis.resize(n);
    for(int i=0;i<n;i++)
      if(!vis[i])dfs1(i);

    vis.assign(n,0);
    for(int i=n-1;i>=0;i--)
      if(!vis[order[i]]){
        comp++;
        dfs2(order[i]);
      }
    adj_cond.resize(comp+1);
    for(int u=0;u<n;u++)
      for(auto v:adj[u])
        if(who[v]!=who[u])
          adj_cond[who[u]].pb(who[v]);
  }

  void dfs1(int u) {
    vis[u] = true;
    for (int v:adj[u])
      if (!vis[v])dfs1(v);
    order.pb(u);
  }

  void dfs2(int u) {
    vis[u] = 1;
    for (int v : adj_t[u])
      if (!vis[v])dfs2(v);
    who[u] = comp;
  }
};
```

3.19 State Space Graph

```
vector<vll> dist(n,vll(1001,INF64));
vector<vector<bool>> vis(n,vector<bool> (1001,0));
typedef array<ll,3> T;
priority_queue<T ,vector<T>,greater<T>> pq;
pq.push({0,0,state[0]});
dist[0][state[0]]= 0;
while(!pq.empty()){
  auto [d,u,s] = pq.top();
  pq.pop();
  if(vis[u][s])continue;
  vis[u][s] = 1;
  for(auto [v,w]:adj[u]){
    ll c  = min(s,slow[v]);
    //Nuevo estad cuando llegue a V
    if(dist[u][s]+w*s<dist[v][c]){
      dist[v][c] = dist[u][s]+w*s;
      pq.push({dist[v][c],v,c});
    }
  }
}
```

3.20 Topological Sort

```
vi ts;
queue<int> q;
for (int i = 1; i < n+1; i++)
  if(!in[i])q.push(i);

while (!q.empty()) {
  int u=q.front();
  q.pop();
  ts.pb(u);
  for (int v : adj[u]) {
    in[v]--;
    if(!in[v])q.push(i);
  }
}

if(sz(ts)!=n)cout <<"IMPOSSIBLE";
```

3.21 Virtual Tree

```
//T[x] tiempo entrada
auto virtualTree = [&](vi &v)->vi{
  sort(all(v),[&](int x,int y){
    return T[x] < T[y];
  });
  int aux = sz(v)-1;
  for(int i=0;i<aux;i++)
    v.pb(lca(v[i],v[i+1]));

  sort(all(v),[&](int x,int y){
    return T[x] < T[y];
  });
  v.erase(unique(all(v)),v.end());

  stack<int> st;
  st.push(v[0]);
  for(int i=1;i<sz(v);i++){
    while(!st.empty() && !isanc(v[i],st.top()))st.pop();
    tree[st.top()].push_back(v[i]);
    st.push(v[i]);
  }
  return v;
};
```

4 Trees

4.1 Centroid Decomposition

```
vector<bool> removed(n+1);
vi sub(n+1);

auto dfs = [&](auto &self,int u,int p)->void{
  sub[u] = 1;
  for(int v:adj[u]){
    if(v!=p && !removed[v]){
      self(self,v,u);
      sub[u] += sub[v];
    }
  }
};

auto centroid = [&](auto &self,int u,int p,int root)->int{
  for(int v:adj[u]){
    if(v!=p && !removed[v]){
      if(sub[v]*2 > sub[root])
        return self(self,v,u,root);
    }
  }
  return u;
};

auto info = [&](auto &self,int u,int p,int d,vi &aux)->void{
  d ^= a[u-1];
  aux.pb(d);
  for(int v:adj[u])
    if(v!=p && !removed[v])
      self(self,v,u,d,aux);
};

auto decompose = [&](auto &self,int u,int p)->void{
  dfs(dfs,u,p);
  int c = centroid(centroid,u,p,u);
  //get info
  removed[c] = 1;
  for(int v:adj[c]){
    if(v!=p && !removed[v])
      self(self,v,c);
  }
};
```

4.2 Link Cut Tree

```
struct Node{
  Node *p = 0, *pp = 0, *c[2];
  bool flip = 0;
  int id = -1,sz = 0;
  Node() { c[0] = c[1] = 0; fix(); }

  void fix(){
    sz = 1;
    if (c[0]){
      c[0]->p = this;
      sz += c[0]->sz;
    }
    if (c[1]){
      c[1]->p = this;
      sz += c[1]->sz;
    }
    // (+ update sum of subtree elements etc. if wanted)
  }

  void pushFlip() {
    if (!flip) return;
    flip = 0; swap(c[0], c[1]);
    if (c[0]) c[0]->flip ^= 1;
    if (c[1]) c[1]->flip ^= 1;
  }

  int up() { return p ? p->c[1] == this : -1; }

  void rot(int i, int b) {
    int h = i ^ b;
    Node *x = c[i], *y = b == 2 ? x : x->c[h], *z = b ? y : x;
    if ((y->p = p)) p->c[up()] = y;
    c[i] = z->c[i ^ 1];
    if (b < 2) {
      x->c[h] = y->c[h ^ 1];
      y->c[h ^ 1] = x;
    }
    z->c[i ^ 1] = this;
    fix(); x->fix(); y->fix();
    if (p) p->fix();
    swap(pp, y->pp);
  }

  void splay() { /// Splay this up to the root. Always finishes
     without flip set.
    for (pushFlip(); p; ) {
      if (p->p) p->p->pushFlip();
      p->pushFlip(); pushFlip();
      int c1 = up(), c2 = p->up();
      if (c2 == -1) p->rot(c1, 2);
      else p->p->rot(c2, c1 != c2);
    }
  }

  Node* first() { /// Return the min element of the subtree rooted
      at this, splayed to the top.
```

```
    pushFlip();
    return c[0] ? c[0]->first() : (splay(), this);
  }
};

struct LCT{
  vector<Node> node;
  LCT(int N):node(N){
    for(int i=0;i<N;i++)node[i].id = i;
  }

  void link(int u,int v){
    makeRoot(&node[u]);
    node[u].pp = &node[v];
  }

  void cut(int u, int v){ // remove an edge (u, v)
    Node *x = &node[u], *top = &node[v];
    makeRoot(top); x->splay();
    assert(top == (x->pp ?: x->c[0]));
    if (x->pp) x->pp = 0;
    else{
      x->c[0] = top->p = 0;
      x->fix();
    }
  }

  void cut(int u){
    access(&node[u]);
    node[u].splay();
    if(node[u].c[0]){
      node[u].c[0]->p = 0;
      node[u].c[0] = 0;
      node[u].fix();
    }
  }

  Node* findRoot(int u){
    return access(&node[u])->first();
  }

  bool connected(int u, int v) { // are u, v in the same tree?
    return findRoot(u)== findRoot(v);
  }

  void makeRoot(Node* u){ /// Move u to root of represented tree.
    access(u);
    u->splay();
    if(u->c[0]){
      u->c[0]->p = 0;
      u->c[0]->flip ^= 1;
      u->c[0]->pp = u;
      u->c[0] = 0;
      u->fix();
    }
  }

  Node* access(Node* u){ /// Move u to root aux tree. Return the
      root of the root aux tree.
    u->splay();

    if(u->c[1]){
      u->c[1]->p = 0;
      u->c[1]->pp = u;
      u->c[1] = 0;
      u->fix();
    }

    while (Node* pp = u->pp){
      pp->splay(); u->pp = 0;
      if (pp->c[1]){
        pp->c[1]->p = 0;
        pp->c[1]->pp = pp;
      }
      pp->c[1] = u; pp->fix(); u = pp;
    }
    return u;
  }

  int depth(int u){
    access(&node[u]);
    node[u].splay();
    node[u].fix();
    return node[u].c[0]->sz;
  }

  Node* lca(int u,int v){
    access(&node[u]);
    return access(&node[v]);
  }
};
```

4.3 Tree Diameter

```
vector<vector<pii>> adj;
vector<bool> vis;

int bfs(int n,vll &d,int v = 0){
  vis.assign(n,0);
  d.assign(n,0);
  d[v] = 0;
  queue<int> q;
  q.push(v);
  pll last = {v,0};
  vis[v] = 1;
  while(!q.empty()){
    int u = q.front();q.pop();
    for(auto [w,c]:adj[u]){
      if(vis[w])continue;
      d[w] = d[u]+c;
      q.push(w);
      vis[w] = 1;
      if(d[w]>last.S)last = {w,d[w]};
    }
  }
  return int(last.F);
}

void solve(){
  int n;cin >> n;
  adj.assign(n,vector<pii> ());
  for(int i=1;i<n;i++){
    int a,b;ll c;
    cin >> a >> b >> c;
    adj[--a].pb({--b,c});
    adj[b].pb({a,c});
  }
  vll dx(n),dy(n);
  int a =bfs(n,dx);
  int b =bfs(n,dy,a);
  bfs(n,dx,b);

  for(int i=0;i<n;i++){
    cout << max(dx[i],dy[i]) << " \n"[i==(n-1)];
  }
}
```

4.4 Euler Tour

```
int timer = 0;
vi start,final;
vector<vi> adj;
void dfs(int u = 0,int prev = -1){
  start[u]=timer++;
  for(int v:adj[u]){
    if(v!=prev)dfs(v,u);
  }
  final[u] = timer;
}
// [start,final)
```

4.5 Heavy Light Decomposition

```
class HLD{
private:
  int n;
  vector<vi> adj;
  vi parent;
  vi heavy;
  vi depth;
  vi root;
  vi treePos;
  Segment_Tree tree;

  int dfs(int u =0){
    int size = 1, mx_sub = 0;
    for(auto v:adj[u])
      if(v!=parent[u]){
        parent[v] = u;
        depth[v] = depth[u]+1;
        int sub = dfs(v);
        if(sub > mx_sub){
          heavy[u] = v;
          mx_sub = sub;
        }
        size += sub;
      }
    return size;
  }

  template <class BinaryOperation>
  void processPath(int u, int v, BinaryOperation op) {
    for (; root[u] != root[v]; v = parent[root[v]]) {
      if (depth[root[u]] > depth[root[v]]) swap(u, v);
      op(treePos[root[v]], treePos[v] + 1);
    }
    if (depth[u] > depth[v]) swap(u, v);
    op(treePos[u], treePos[v] + 1);
  }

  void init(vi &a){
    for(int i=0,crt=0;i<n;++i){
      if(parent[i] == -1 || heavy[parent[i]]!=i){
        for(int j = i;j!=-1;j = heavy[j]){
          root[j] = i;
          treePos[j] = crt++;
        }
      }
    }
    tree = Segment_Tree(n);
    for(int i=0;i<n;i++){
      tree.update(treePos[i],a[i]);
```

```
    }
  }

public:

  HLD(int n,vector<vi> &vale,vi &a):n(n),adj(vale){
    parent.assign(n,0);
    heavy.assign(n,-1);
    depth.assign(n,0);
    root.resize(n);
    treePos.assign(n,0);
    parent[0] = -1;
    dfs();
    init(a);
  }

  void update(int u,int &val){
    tree.update(treePos[u],val);
  }

  void updatePath(int u, int v, int value) {
    processPath(u, v, [this, &value](int l, int r) {
      tree.update_range(l, r, value);
    });
  }

  int query(int u,int v){
    int ans = 0;
    processPath(u,v,[this,&ans](int l,int r){
      ans = max(ans,tree.query(l,r));
    });
    return ans;
  }
};
```

4.6 LCA

```
const int LG = 18;

vi deep(n+1);
vector<vi> par(n+1,vi(LG+1));

auto dfs = [&](auto &self,int u,int p)->void{
  par[u][0] = p;
  deep[u] = deep[p]+1;//cost[u] += cost[p]
  for(int i=1;i<=LG;i++)par[u][i]=par[par[u][i-1]][i-1];
  for(int v:adj[u])
    if(v!=p)
      self(self,v,u);
};

auto lca = [&](int u,int v)->int{
  if(deep[u] < deep[v])swap(u,v);
  for(int k=LG;k>=0;k--)
    if(deep[par[u][k]] >= deep[v])
      u = par[u][k];
  if(u==v)return u;
  for(int k=LG;k>=0;k--)
    if(par[u][k]!=par[v][k])
      u=par[u][k],v=par[v][k];
  return par[u][0];
};

auto dist = [&](int u,int v)->int{
  int lc = lca(u,v);
  return deep[u]+deep[v]-(deep[lc]<<1);
};

```

```
auto kth = [&](int u, int k)->int{
  assert(k>=0);
  for(int i=0;i<=LG;i++)
    if(k&(1<<i))u = par[u][i];
  return u;
};

auto isanc = [&](int u,int g)->bool{
  int k = deep[u]-deep[g];
  return k >= 0 && kth(u, k) == g;
};
```

4.7 Min/Max Weight on Path

```
const int MAX = 1e5+5,LG=18;
vector<vector<pii>> adj(MAX,vector<pii> ());
vi deep(MAX);
int par[MAX][LG+1];
int mn[MAX][LG+1];
int mx[MAX][LG+1];

void dfs(int u=1,int p=0,int costmn = INF,int costmx = -INF){
  par[u][0] = p;
  deep[u] = deep[p]+1;

  if(p!=0){
    mn[u][0]  = costmn;
    mx[u][0]  = costmx;
  }
  for(int i=1;i<=LG;i++){
    par[u][i]=par[par[u][i-1]][i-1];
    mn[u][i]=min(mn[u][i-1],mn[par[u][i-1]][i-1]);
    mx[u][i]=max(mx[u][i-1],mx[par[u][i-1]][i-1]);
  }

  for(auto [v,t]:adj[u]){
    if(v!=p){
      dfs(v,u,t,t);
    }
  }
}

pii minmax(int u,int v){
  pii ans = {INF,-INF};
  if(deep[u] < deep[v])swap(u,v);
  for(int k=LG;k>=0;k--)if(deep[par[u][k]] >= deep[v]){
    ans.F = min(ans.F,mn[u][k]);
    ans.S = max(ans.S,mx[u][k]);
    u = par[u][k];
  }
  if(u==v)return ans;
  for(int k=LG;k>=0;k--)if(par[u][k]!=par[v][k]){
    ans.F = min({ans.F,mn[u][k],mn[v][k]});
    ans.S = max({ans.S,mx[u][k],mx[v][k]});
    u=par[u][k],v=par[v][k];
  }
  ans.F = min({ans.F,mn[u][0],mn[v][0]});
  ans.S = max({ans.S,mx[u][0],mx[v][0]});

  return ans;
}
```

4.8 Intersection Path

```
pair<int, int> getCommonPath(int u, int a, int v, int b) {
// returns common path between u->a and v->b where u is ancestor a
      and v is ancestor b
  if (!isanc(a, v)) return make_pair(0, 0);
  int x = lca(a, b);
  if (dep[v] < dep[u]) {
    if (isanc(x, u))
      return make_pair(u, x);
  }else{
    if (isanc(x, v))
      return make_pair(v, x); // v->x is the common path
  }
  return make_pair(0, 0);
}
pair<int, int> path_union_light(pair<int, int> x, pair<int, int> y
    ) {
  if (x.first == 0) return y; //where {0,0} is a null path
  if (y.first == 0) return x;
  vector<int> can = {x.first, x.second, y.first, y.second};
  int a = can[0];
  for (int u : can) if (dep[u] > dep[a]) a = u;
  int b = -1;
  for (int u : can)if (!isanc(a, u)) {
    if (b == -1) b = u;
    if (dep[b] < dep[u]) b = u;
  }
  if (b == -1) {
    b = can[0];
    for (int u : can) if (dep[u] < dep[b]) b = u;
    return {a, b};
  }
  return {a, b};
}
// returns the intersection of two paths (a,b) and (c,d)
// {0,0} for null path
pair<int, int> path_intersection(int a, int b, int c, int d) {
  if (a == 0 || b == 0) return {0, 0};
  int u = lca(a, b), v = lca(c, d); // splits both paths in two
      chains from lca to one node
  pair<int, int> x, y;
  x = getCommonPath(u, a, v, c);
  y = getCommonPath(u, a, v, d);
  pair<int, int> a1 = path_union_light(x, y);
  x = getCommonPath(u, b, v, c);
  y = getCommonPath(u, b, v, d);
  pair<int, int> a2 = path_union_light(x, y);
  return path_union_light(a1, a2);
}
```

4.9 Union Path

```
pair<int, int> merge(pair<int, int> x, pair<int, int> y) {
  if (x.first == 0)return y; //where {0,0} is a null path
  if (y.first == 0)return x;
  if (x.first == -1 || y.first == -1)   return { -1, -1};
  vector<int> can = {x.first, x.second, y.first, y.second};
  int a = can[0];
  for (int u : can)
    if (dep[u] > dep[a])
      a = u;
  int b = -1;
  for (int u : can)
    if (!isanc(a, u)) {
      if (b == -1)  b = u;
      if (dep[b] < dep[u])    b = u;
    }
  if (b == -1) {
    b = can[0];
    for (int u : can)
      if (dep[u] < dep[b])
        b = u;
    return {a, b};
```

```
  }
  int g = lca(a, b);
  for (int u : can) {
    if (u == a || u == b)continue;
    if (dep[u] < dep[g] || (!isanc(a, u) && !isanc(b, u)))return {
      -1, -1};
  }
  return {a, b};
}
```

4.10 Prufer Code Build

```
auto dfs = [&](auto &self,int u){
    for (int v:adj[u]) {
        if (v != parent[u]) {
            parent[v] = u;
            dfs(v);
        }
    }
};

parent[n] = -1;
dfs(dfs,n);

int ptr = -1;
vi degree(n);
for(int i=1;i<=n;i++) {
        degree[i] = sz(adj[i]);
        if(degree[i]==1 && ptr==-1)
    ptr = i;
}

vi code(n - 2);
int leaf = ptr;
for(int i=0;i<n-2;i++){
  int next = parent[leaf];
  code[i] = next;
  if (--degree[next]==1 && next<ptr) {
    leaf = next;
  }else{
    ptr++;
    while (degree[ptr] != 1)
      ptr++;
    leaf = ptr;
  }
}
```

4.11 Prufer Code Restore

```
int n = code.size() + 2;
vi degree(n,0);
for (int i : code)
  degree[i]++;

int ptr = 0;
while (degree[ptr] != 1)
  ptr++;

int leaf = ptr;

vector<pii> edges;
for(int v:code){
  edges.emplace_back(leaf, v);
  if (--degree[v] == 0 && v < ptr) {
    leaf = v;
  }else{
    ptr++;
    while (degree[ptr] != 1)
      ptr++;
    leaf = ptr;
  }
}
edges.emplace_back(leaf, n-1);
```

4.12 Tree Rerooting

```
namespace reroot{
  const auto exclusive = [](const auto& a, const auto& base, const
       auto& merge_into, int vertex){
    int n = (int)a.size();
    using Aggregate = decay_t<decltype(base)>;
    vector<Aggregate> b(n,base);
    for(int bit=(int)__lg(n); bit>=0;--bit){
      for(int i=n-1;i>=0;--i)b[i]=b[i >> 1];
      int sz=n-(n&!bit);
      for(int i=0;i<sz;++i){
        int index = (i >> bit) ^ 1;
        b[index] = merge_into(b[index], a[i], vertex, i);
      }
    }
    return b;
  };
  // MergeInto : Aggregate * Value * Vertex(int) * EdgeIndex(int)
     -> Aggregate
  // Base : Vertex(int) -> Aggregate
  // FinalizeMerge : Aggregate * Vertex(int) * EdgeIndex(int) ->
     Value
  const auto rerooter = [](const auto& g, const auto& base, const
     auto& merge_into, const auto& finalize_merge){
    int n = (int)g.size();
    using Aggregate = decay_t<decltype(base(0))>;
    using Value = decay_t<decltype(finalize_merge(base(0), 0, 0))
     >;
    vector<Value> root_dp(n), dp(n);
    vector<vector<Value>> edge_dp(n), redge_dp(n);

    vector<int> bfs, parent(n);
    bfs.reserve(n);
    bfs.push_back(0);
    for(int i=0;i<n;++i){
      int u = bfs[i];
      for (auto v : g[u]) {
        if (parent[u] == v) continue;
        parent[v] = u;
        bfs.push_back(v);
      }
    }

    for(int i=n-1;i>=0;--i){
      int u = bfs[i];
      int p_edge_index = -1;
      Aggregate aggregate = base(u);
      for(int edge_index = 0; edge_index < (int)g[u].size(); ++
     edge_index){
        int v = g[u][edge_index];
        if(parent[u] == v) {
          p_edge_index = edge_index;
          continue;
        }
        aggregate = merge_into(aggregate, dp[v], u, edge_index);
      }
      dp[u] = finalize_merge(aggregate, u, p_edge_index);
    }

    for (auto u : bfs) {
      dp[parent[u]] = dp[u];
      edge_dp[u].reserve(g[u].size());
      for (auto v : g[u]) edge_dp[u].push_back(dp[v]);
      auto dp_exclusive = exclusive(edge_dp[u], base(u),
     merge_into, u);
      redge_dp[u].reserve(g[u].size());
      for (int i = 0; i < (int)dp_exclusive.size(); ++i) redge_dp[
     u].push_back(finalize_merge(dp_exclusive[i], u, i));
      root_dp[u] = finalize_merge(n > 1 ? merge_into(dp_exclusive
     [0], edge_dp[u][0], u, 0) : base(u), u, -1);
      for (int i = 0; i < (int)g[u].size(); ++i) {
        dp[g[u][i]] = redge_dp[u][i];
      }
    }

    return make_tuple(move(root_dp), move(edge_dp), move(redge_dp)
     );
  };
}

//in solve()
  using Aggregate = int;
  using Value = int;

  auto base = [&](int vertex) -> Aggregate {
    //base case
    return ;
  };

  auto merge_into = [](Aggregate vertex_dp, Value neighbor_dp, int
       vertex, int edge_index) -> Aggregate {
    //merge childs
    return vertex_dp;
  };

  auto finalize_merge = [&](Aggregate vertex_dp, int vertex, int
     edge_index) -> Value {
    //return of dfs
    return vertex_dp;
  };

  auto [root_dp, edge_dp, redge_dp] = reroot::rerooter(adj, base,
     merge_into, finalize_merge);
```

4.13 Tree Matching

```
vector<vll> dp(2,vll(MAX));

void dfs(ll u = 0, ll p=-1){
  dp[0][u] = 0;
  dp[1][u] = -INF;

  for(ll v:adj[u]){
    if(v == p)continue;
    dfs(v,u);

    dp[0][u]+=max(dp[1][v],dp[0][v]);

    ll opt = min(dp[0][v]-dp[1][v],0ll);

    dp[1][u] = max(dp[1][u],opt);
  }

  dp[1][u] += dp[0][u];
  dp[1][u]++;
}
```

4.14 Rooted Tree Hashing

```
auto dfs = [](auto &self,int u,int p,vector<vi> &adj)->pair<ll,ll
    >{
  vector<pair<ll,ll>> shash;
  for(int v:adj[u]){
    if(v==p)continue;
    shash.pb(self(self,v,u,adj));
  }
  sort(all(shash));

  ll p1 = P1,p2 = P2;
  pair<ll,ll> hash = {42,42};
  for(int i=0;i<sz(shash);i++,(p1*=p1)%=MOD1,(p2*=p2)%=MOD2){
    (hash.F += shash[i].F*shash[i].F%MOD1+shash[i].F*p1%MOD1+42)%
     MOD1;
    (hash.S += shash[i].S*shash[i].S%MOD2+shash[i].S*p2%MOD2+42)%
     MOD2;
  }
  return hash;
};
```

4.15 Tree Isomorphism

```
//1-indexed

auto centers = [&](vector<vi> &adj)->vi{
  //Diameters of a tree
  vll du(n+1),dv(n+1);
  int v =bfs(du,1,adj);
  int u =bfs(dv,v,adj);
  bfs(du,u,adj);
  vi ans;
  for(int i=1;i<=n;i++)
    if(du[i]+dv[i]==du[v] && du[i]>=du[v]/2 && dv[i]>=du[v]/2)
      ans.pb(i);
  return ans;
};

//in main

vi a = centers(tree1);
vi b = centers(tree2);

for(auto x:a)
  for(auto y:b)
    //checar isomorfismos si estan rooteados
    //en x y y
```

5 Number Theory

5.1 Euler Totient

```

vi phi(N);
iota(all(phi),0);

for(int i=2;i<N;i++)
  if(phi[i]==i)
    for(int j=i;j<N;j+=i)
      phi[j] -= phi[j]/i
```

5.2 Mobius Function

```
int mob[N];

mob[1] = 1;
for(int i=2;i<N;i++){
  mob[i]--;
  for(int j=i+i;j<N;j+=i)
    mob[j] -= mob[i];
}
```

5.3 Chinese Reminder Theorem

```
using T = __int128;
// ax + by = __gcd(a, b)
// returns __gcd(a, b)
T extended_euclid(T a, T b, T &x, T &y){
  T xx = y = 0;
  T yy = x = 1;
  while(b){
    T q = a / b;
    T t = b; b = a % b; a = t;
    t = xx; xx = x - q * xx; x = t;
    t = yy; yy = y - q * yy; y = t;
  }
  return a;
}
// finds x such that x % m1 = a1, x % m2 = a2. m1 and m2 may not
     be coprime
// here, x is unique modulo m = lcm(m1, m2). returns (x, m). on
     failure, m = -1.
pair<T, T> CRT(T a1, T m1, T a2, T m2){
  T p, q;
  T g = extended_euclid(m1, m2, p, q);
  if (a1 % g != a2 % g) return make_pair(0, -1);
  T m = m1 / g * m2;
  p = (p%m + m)%m;
  q = (q%m + m)%m;
  return make_pair((p * a2 % m * (m1 / g) % m + q * a1 % m * (m2 /
      g) % m) %  m, m);
}
```

5.4 Binpow

```
ll binpow(ll a,ll b){
  a%=MOD;
  ll ans = 1;
  while(b){
    if(b&1)(ans*=a)%=MOD;
    (a*=a)%=MOD;
    b>>=1;
  }
  return ans;
}
```

6 Combinatorics

6.1 nCr

```
// Hash: 2e457c
const int MAX = 2e6;
ll factorial[MAX+1];
ll inv[MAX+1]; // #d459d3

ll binpow(ll a,ll b){
  ll res = 1;
  while(b){
    if(b&1)res = res*a%MOD;
    a = a*a%MOD;
    b>>=1;
  }
  return res;
} // #89684f

void precalc(){
  factorial[0]=1;
  for (ll i = 1; i <= MAX; i++) {
    factorial[i] = factorial[i-1]*i%MOD;
  }
  inv[MAX] = binpow(factorial[MAX],MOD-2);
  for (ll i = MAX; i >= 1; i--) {
    inv[i-1] = inv[i]*i%MOD;
  }
} // #c4e465

ll ncr(ll a,ll b){
  return factorial[a]*inv[b]%MOD*inv[a-b]%MOD;
} // #f2f007
```

7 Math

7.1 FFT

```
typedef complex<double> base;

void fft(vector<base> &p, bool inv = 0) {
  int n = p.size(), i = 0;
  for(int j = 1; j < n - 1; ++j) {
    for(int k = n >> 1; k > (i ^= k); k >>= 1);
    if(j < i) swap(p[i], p[j]);
  }

  for(int l = 1, m; (m = l << 1) <= n; l <<= 1) {
    double ang = 2 * PI / m *(inv?1.0:-1.0);
    base wn(cos(ang),sin(ang));
    for(int i = 0; i < n; i += m) {
      base w(1,0);
      for(int j = i, k = i + l; j < k; ++j, w *= wn) {
        base t = w * p[j + l];
        p[j + l] = p[j] - t;
        p[j] = p[j] + t;
      }
    }
  }

  if(inv) for(int i = 0; i < n; ++i) p[i] /= n;
}
vector<ll> multiply(vi &a, vi &b) {
  int n = a.size(), m = b.size(), t = n + m - 1, sz = 1;
  while(sz < t) sz <<= 1;
  vector<base> x(sz), y(sz);
  for(int i = 0; i < n; ++i) x[i] = base(a[i], 0);
  for(int i = 0; i < m; ++i) y[i] = base(b[i], 0);
  fft(x), fft(y);
  for(int i = 0; i < sz; ++i) x[i] *= y[i];
  fft(x, 1);
  vector<ll> ret(sz);
  for(int i = 0; i < sz; ++i) ret[i] = (ll)round(x[i].real());
  while((int)ret.size() > 1 && ret.back() == 0) ret.pop_back();
  return ret;
}
```

7.2 FWDT

```
const int MOD = 1e9+7;
const int inv2 = (MOD + 1) >> 1;
#define M (1 << 20)
#define OR 0
#define AND 1
#define XOR 2

int P1[M],P2[M];
void wt(int *a, int n, int flag = XOR){
  if (n == 0) return;
  int m = n / 2;
  wt(a, m, flag); wt(a + m, m, flag);
  for(int i = 0; i < m; i++){
    int x = a[i], y = a[i + m];
    if (flag == OR) a[i] = x, a[i + m] = (x + y) % MOD;
```

```
    if (flag == AND) a[i] = (x + y) % MOD, a[i + m] = y;
    if (flag == XOR) a[i] = (x + y) % MOD, a[i + m] = (x - y + MOD
      ) % MOD;
  }
}
void iwt(int* a, int n, int flag = XOR) {
  if (n == 0) return;
  int m = n / 2;
  iwt(a, m, flag); iwt(a + m, m, flag);
  for(int i = 0; i < m; i++){
  int x = a[i], y = a[i + m];
    if (flag == OR) a[i] = x, a[i + m] = (y - x + MOD) % MOD;
    if (flag == AND) a[i] = (x - y + MOD) % MOD, a[i + m] = y;
    if (flag == XOR) a[i] = 1LL * (x + y) * inv2 % MOD, a[i + m] =
       1LL * (x - y + MOD) * inv2 % MOD; // replace inv2 by >>1 if
      not required
  }
}

vector<int> multiply(int n, vector<int> &A, vector<int> &B, int
     flag = XOR) {
  assert(__builtin_popcount(n) == 1);
  A.resize(n); B.resize(n);
  for(int i = 0; i < n; i++) P1[i] = A[i];
  for(int i = 0; i < n; i++) P2[i] = B[i];
  wt(P1, n, flag); wt(P2, n, flag);
  for (int i = 0; i < n; i++) P1[i] = 1LL * P1[i] * P2[i] % MOD;
  iwt(P1, n, flag);
  return vector<int> (P1, P1 + n);
}
```

7.3 NTT

```
void ntt(vll &a){
  int n = sz(a), L = 31 - __builtin_clz(n);
  static vll rt(2, 1);
  for (static int k = 2, s = 2; k < n; k *= 2, s++) {
    rt.resize(n);
    ll z[] = {1, binpow(root, MOD >> s)};
    for(int i=k;i<2*k;i++)
      rt[i] = rt[i / 2] * z[i & 1] % MOD;
  }
  vi rev(n);
  for(int i=0;i<n;i++) rev[i] = (rev[i / 2] | (i & 1) << L) / 2;
  for(int i=0;i<n;i++) if (i < rev[i]) swap(a[i], a[rev[i]]);
  for (int k = 1; k < n; k *= 2)
    for (int i = 0; i < n; i += 2 * k)
      for(int j=0;j<k;j++){
      ll z = rt[j + k] * a[i + j + k] % MOD, &ai = a[i + j];
      a[i + j + k] = ai - z + (z > ai ? MOD : 0);
      ai += (ai + z >= MOD ? z - MOD : z);
    }
}

vll conv(const vll &a, const vll &b){
  if (a.empty() || b.empty()) return {};
  int s = sz(a) + sz(b) - 1, B = 32 - __builtin_clz(s),
      n = 1 << B;
  int inv = binpow(n, MOD - 2);
  vll L(a), R(b), out(n);
  L.resize(n), R.resize(n);
  ntt(L), ntt(R);
  for(int i=0;i<n;i++)
    out[-i & (n - 1)] = (ll)L[i] * R[i] % MOD * inv % MOD;
  ntt(out);
  return {out.begin(), out.begin() + s};
}
```

7.4 FFT Any Mod

```
template<int M> vll convMod(const vll &a, const vll &b) {
  if(a.empty() || b.empty()) return {};
  vll res(sz(a) + sz(b) - 1);
  int B = 32 - __builtin_clz(sz(res)), n = 1 << B, cut = int(sqrt(
     M));
  vector<base> L(n), R(n), outs(n), outl(n);
  for(int i = 0; i < sz(a); i++) L[i] = base((int)a[i] / cut, (int
     )a[i] % cut);
  for(int i = 0; i < sz(b); i++) R[i] = base((int)b[i] / cut, (int
     )b[i] % cut);
  fft(L), fft(R);
  for(int i = 0; i < n; i++) {
    int j = -i & (n - 1);
    outl[j] = (L[i] + conj(L[j])) * R[i] / (2.0 * n);
    outs[j] = (L[i] - conj(L[j])) * R[i] / (2.0 * n) / 1i;
  }
  fft(outl), fft(outs);
  for(int i = 0; i < sz(res); i++) {
    ll av = ll(real(outl[i]) + .5), cv = ll(imag(outs[i]) + .5);
    ll bv = ll(imag(outl[i]) + .5) + ll(real(outs[i]) + .5);
    res[i] = ((av % M * cut + bv) % M * cut + cv) % M;
  }
  return res;
}
```

7.5 Or Convolution

```
void solve(){
int n,k; cin>>n>>k;
vector<ll> acu(1<<k), sos(1<<k);
string s;
for(int i=0; i<n ; i++){
cin>>s;
sos[stoi(s,nullptr, 2)]++;
}
for (int i = 0; i < k; i++) {
for (int j = 0; j < (1 << k); j++) {
if ((j >> i) & 1) {
sos[j] += sos[j - (1 << i)];
}
}
}
for(int i=0; i<(1<<k); i++) {
if(i<3) acu[i]=0;
acu[i] = ((sos[i])*(sos[i]-1)*(sos[i]-2))/6;
}
for (int i = k - 1; i >= 0; i--) {
for (int j = (1 << k) - 1; j >= 0; j--) {
if ((j >> i) & 1) {
acu[j] -= acu[j - (1 << i)];
}
}
}
int m; cin>>m;
for(int i=0; i<m; i++){
cin>>s;
cout<<acu[stoi(s,nullptr, 2)]<<endl;
}
}
```

7.6 Xor Basis

```
template<typename T = int, int B = 31>
struct Basis{
  T basis[B]; int sz;
  void clear() {
    memset(basis, 0, sizeof basis);
    sz = 0;
  }
  Basis(){clear();}

  void insert(T x) {
    for(int i=B-1;i>=0;i--){
      if(x>>i&1){
        if(basis[i])x ^= basis[i];
        else{
          basis[i] = x;
          sz++;
          break;
        }
      }
    }
  }

  bool can(T x){
    for(int i=B-1;i>=0;i--)
      x = min(x, x ^ basis[i]);
    return x == 0;
  }

  T kth(T k){
    T x = 0;
    T cnt  = ((T)1<<sz);
    for(int i=B-1;i>=0;i--){
      if(!basis[i])continue;
      if(k>(cnt>>1)){
        if (!(x>>i&1))x ^= basis[i];
        k -= (cnt>>1);
      }else{
        if (x>>i&1)x ^= basis[i];
      }
      cnt>>=1;
    }
    return x;
  }

  bool merge(Basis &b){
    bool flag = false;
    for(ll i=B-1;i>=0;i--)
      if(b.basis[i] && !insert(b.basis[i])) flag = true;
    return flag;
  }

};
```

7.7 Matrix

```
struct Mat {
  int n, m;
  vector<vector<int>> a;
  Mat() { }
  Mat(int _n, int _m) {n = _n; m = _m; a.assign(n, vector<int>(m,
     0)); }
  Mat(vector< vector<int> > v) { n = v.size(); m = n ? v[0].size()
      : 0; a = v; }
  inline void make_unit() {
    assert(n == m);
    for (int i = 0; i < n; i++)  {
      for (int j = 0; j < n; j++) a[i][j] = i == j;
    }
  }
  inline Mat operator + (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
```

```
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] + b.a[i][j]) % mod;
      }
    }
    return ans;
  }
  inline Mat operator - (const Mat &b) {
    assert(n == b.n && m == b.m);
    Mat ans = Mat(n, m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < m; j++) {
        ans.a[i][j] = (a[i][j] - b.a[i][j] + mod) % mod;
      }
    }
    return ans;
  }
  inline Mat operator * (const Mat &b) {
    assert(m == b.n);
    Mat ans = Mat(n, b.m);
    for(int i = 0; i < n; i++) {
      for(int j = 0; j < b.m; j++) {
        for(int k = 0; k < m; k++) {
          ans.a[i][j] = (ans.a[i][j] + 1LL * a[i][k] * b.a[k][j] %
     mod) % mod;
        }
      }
    }
    return ans;
  }
  inline Mat pow(long long k) {
    assert(n == m);
    Mat ans(n, n), t = a; ans.make_unit();
    while (k) {
      if (k & 1) ans = ans * t;
      t = t * t;
      k >>= 1;
    }
    return ans;
  }
  inline Mat& operator += (const Mat& b) { return *this = (*this)
     + b; }
  inline Mat& operator -= (const Mat& b) { return *this = (*this)
     - b; }
  inline Mat& operator *= (const Mat& b) { return *this = (*this)
     * b; }
  inline bool operator == (const Mat& b) { return a == b.a; }
  inline bool operator != (const Mat& b) { return a != b.a; }
};

// Usage
// Mat a(n, n);
// Mat b(n, n);
// Mat c = a * b;
// Mat d = a + b;
// Mat e = a - b;
// Mat f = a.pow(k);
// a.a[i][j] = x;
```

7.8 Gauss

```
typedef vector<double> vd;
const double eps = 1e-12;

int solveLinear(vector<vd>& A, vd& b, vd& x){
  int n = sz(A), m = sz(x), rank = 0, br, bc;
  if (n) assert(sz(A[0]) == m);
  vi col(m); iota(all(col), 0);

  for(int i=0;i<n;i++){
    double v, bv = 0;
    for(int r=i;r<n;r++)
      for(int c = i;c<m;c++)
        if ((v = fabs(A[r][c])) > bv)
          br = r, bc = c, bv = v;
    if (bv <= eps){
      for(int j=i;j<n;j++)
        if (fabs(b[j]) > eps) return -1;
      break;
    }
    swap(A[i], A[br]);
    swap(b[i], b[br]);
    swap(col[i], col[bc]);
    for(int j=0;j<n;j++)swap(A[j][i], A[j][bc]);
    bv = 1/A[i][i];
    for(int j=i+1;j<n;j++){
      double fac = A[j][i] * bv;
      b[j] -= fac * b[i];
      rep(k,i+1,m) A[j][k] -= fac*A[i][k];
    }
    rank++;
  }

  x.assign(m, 0);
  for (int i = rank; i--;) {
    b[i] /= A[i][i];
    x[col[i]] = b[i];
    rep(j,0,i) b[j] -= A[j][i] * b[i];
  }
  return rank; // (multiple solutions if rank < m)
}
```

7.9 Gauss MOD

```
int gauss (vector < vector<ll> > a, vector<ll> & ans){
  int n = (int) a.size();
  int m = (int) a[0].size() - 1;

  vector<int> where (m, -1);
  for (int col=0, row=0; col<m && row<n; ++col) {
    int sel = row;
    for (int i=row; i<n; ++i)
      if(abs (a[i][col]) > abs (a[sel][col]))
        sel = i;

    if (a[sel][col] == 0)continue;
    for (int i=col; i<=m;++i)swap (a[sel][i], a[row][i]);
    where[col] = row;

    for (int i=0; i<n; ++i)
      if (i != row){
        ll c = divide(a[i][col] , a[row][col]);
        for (int j=col; j<=m; ++j)
          a[i][j] = (a[i][j]-a[row][j] * c % MOD + MOD) % MOD;
      }
    ++row;
  }

  ans.assign(m, 0);
  for (int i=0; i<m; ++i)if (where[i] != -1)
           ans[i] = divide(a[where[i]][m] , a[where[i]][i]);

  for (int i=0; i<n; ++i){
    ll sum = 0;
    for(int j=0; j<m; ++j)
      sum = (sum + ans[j] * a[i][j] % MOD) % MOD;
    if (sum - a[i][m])return 0;
  }

  for(int i=0; i<m; ++i)
    if (where[i] == -1)
      return 2;
  return 1;
}
```

8 Strings

8.1 Phi Function

```
vi pi(n,0);
for(int i=1;i<n;i++){
  int j = pi[i-1];
  while(j && s[i]!=s[j])j = pi[j-1];
  if(s[i] == s[j])j++;
  pi[i] = j;
}
```

8.2 Z Function

```
// greatest number of characters starting from pos i
// that coincide with the first characters of s

vi z(n,0);
int l = 0,r = 0;
for(int i=1;i<n;i++){
  if(i<r)z[i] = min(r-i,z[i-l]);

  while(i+z[i]<n && s[z[i]] == s[i+z[i]])z[i]++;

  if(i+z[i]>r){
    l = i;
    r = i+z[i];
  }
}
```

8.3 Aho Corasick

```
struct AC{
  static const int K = 26;
  struct Node{
    int ch[K];
    int fail;
    int last_m;
    vi id;
    Node(){
      memset(ch,0,sizeof(ch));
      fail = last_m = 0;
    }
  };

  vector<Node> t;

  AC(){t.emplace_back();}

  void insert(string& s, int id){
    int u = 0;
    for(char c:s){
      int v = c-'a';
      if(!t[u].ch[v]){
        t[u].ch[v] = sz(t);
        t.emplace_back();
      }
      u = t[u].ch[v];
    }
    t[u].id.push_back(id);
  }
```

```

  void build(){
    queue<int> q;
    for(int i=0;i<K;i++)
      if (t[0].ch[i]) q.push(t[0].ch[i]);

    while(!q.empty()){
      int u=q.front();q.pop();
      int f = t[u].fail;
      t[u].last_m = t[f].id.empty() ? t[f].last_m:f;

      for(int i=0;i<K;i++){
        int &v = t[u].ch[i];
        if(v){
          t[v].fail = t[f].ch[i];
          q.push(v);
        }else v = t[f].ch[i];
      }
    }
  }

  int count(string &s){
    ll ans = 0;
    int u = 0;
    for(char c:s){
      u = t[u].ch[c - 'a'];
      int temp = u;
      while(temp > 0){
        ans += t[temp].id.size();
        temp = t[temp].last_m;
      }
    }
    return ans;
  }
};
```

8.4 Dynamic Aho Corasick

```
vector<AC> a(19);// log aho corasick
vector<vector<string>> ws;
int cnt = 0;//number of strings insert

auto insert = [&](string &s){
  int pos = 0;
  for(int i=0;i<19;i++)
    if(ws[i].empty()){
      pos = i;
      break;
    }

  ws[pos].pb(s);
  for(int i=0;i<pos;i++){
    for(auto &str:ws[i])
      ws[pos].pb(move(str));
    ws[i].clear();
    w[i] = AC();
  }

  w[pos] = AC();

  for(int i=0;i<sz(ws[pos]);i++)w[pos].insert(ws[pos][i],i+1);

  w[pos].build();

};

auto query = [&](string &s)->ll{
  ll ans = 0;
  for(int i=0;i<19;i++)
    if(cnt>>i&1)
      ans += a[i].count(s);
};
```

8.5 Suffix Array

```
struct SuffixArray{
  vi sa, lcp;
  SuffixArray(string s, int lim=256) { // or vector<int>
    s.push_back(0); int n = sz(s), k = 0, a, b;
    vi x(all(s)), y(n), ws(max(n, lim));
    sa = lcp = y, iota(all(sa), 0);
    for (int j = 0, p = 0; p < n; j = max(1, j * 2), lim = p) {
      p = j, iota(all(y), n - j);
      for(int i=0;i<n;i++)if (sa[i] >= j) y[p++] = sa[i] - j;
      fill(all(ws), 0);
      for(int i=0;i<n;i++)ws[x[i]]++;
      for(int i=1;i<lim;i++) ws[i] += ws[i - 1];
      for (int i = n; i--;) sa[--ws[x[y[i]]]] = y[i];
      swap(x, y), p = 1, x[sa[0]] = 0;
      for(int i=1;i<n;i++){
        a = sa[i - 1];
        b = sa[i];
        x[b] = (y[a] == y[b] && y[a + j] == y[b + j]) ? p - 1 : p
      ++;
      }
    }
    for (int i = 0, j; i < n - 1; lcp[x[i++]] = k)
      for (k && k--, j = sa[x[i] - 1];
          s[i + k] == s[j + k]; k++);
  }
};
```

8.6 Hashing

```
ll binpow(ll n,ll k,const int mod){
  ll res = 1;
  n%=mod;
  while(k){
    if(k&1)(res*=n)%=mod;
    (n*=n)%=mod;
    k>>=1;
  }
  return res;
}

const int MOD1 = 127657753,MOD2 = 987654319;
const int P1 = 137,P2 = 277;
const int MAX = 1e6+9;

vector<pll> P(MAX),I_P(MAX);

void calc(){
  P[0] = {1,1};
  for(int i=1;i<MAX;i++){
    P[i].F = P[i-1].F*P1%MOD1;
    P[i].S = P[i-1].S*P2%MOD2;
  }
  int i1 = binpow(P1,MOD1-2,MOD1);
  int i2 = binpow(P2,MOD2-2,MOD2);
  I_P[0] = {1,1};
  for(int i=1;i<MAX;i++){
    I_P[i].F = I_P[i-1].F*i1%MOD1;
    I_P[i].S = I_P[i-1].S*i2%MOD2;
  }
}

struct Hashing {
  int n;
  string s; // 0 - indexed
  vector<pll> hs; // 1 - indexed

  Hashing(string _s) {
    n = _s.size();
    s = _s;
    hs.emplace_back(0, 0);
    for (int i = 0; i < n; i++) {
      pll p;
      p.F = (hs[i].F + P[i].F * (s[i]-'a'+1) % MOD1) % MOD1;
      p.S = (hs[i].S + P[i].S * (s[i]-'a'+1) % MOD2) % MOD2;
      hs.push_back(p);
    }
  }

  pll get_hash(int l, int r) { // 1 - indexed
    pll ans;
    ans.F = (hs[r].F - hs[l - 1].F+ MOD1) * 1LL * I_P[l - 1].F%
      MOD1;
    ans.S = (hs[r].S - hs[l - 1].S+ MOD2) * 1LL * I_P[l - 1].S%
      MOD2;
    return ans;
  }
};
```

8.7 Palindromic Tree

```
struct Eertree{
  struct Node{
    int nxt[26],len,st,en,link,cnt,oc;
  };//f0c5da

  vector<Node> t;
  string s;
  int sz, last;
  Eertree(){}//24d5fc

  Eertree(string _s){
    s = _s;
    int n = sz(s);
    t.resize(n+5);
    sz = 2, last = 2;
    t[1].len = -1, t[1].link = 1;
    t[2].len = 0 , t[2].link = 1;
  }//1fe603

  int get_link(int u,int pos){
    while(pos-1-t[u].len<0 || s[pos-1-t[u].len] != s[pos])
      u = t[u].link;
    return u;
  }//72a63c

  bool add(int pos){
    int ch = s[pos]-'a';
    int cur = get_link(last,pos);

    if(t[cur].nxt[ch]){
      last = t[cur].nxt[ch];
      t[last].oc++;
      return 0;
    }//4c7cf1

    sz++;
    t[sz].len = t[cur].len+2;
    t[sz].en = pos;
    t[sz].st = pos-t[sz].len+1;//3b0156
```

```
    if(t[sz].len == 1)t[sz].link = 2;
    else t[sz].link = t[get_link(t[cur].link,pos)].nxt[ch];

    t[sz].cnt = t[t[sz].link].cnt+1;
    t[sz].oc = 1;
    t[cur].nxt[ch] = sz;
    last = sz;
    return 1;//e1a942
  }//4b0fbf

  void calc_occurrences(){
    for(int i=sz;i>2;i--)
      t[t[i].link].oc += t[i].oc;
  }//38a897

  void show(){
    for(int i=3;i<=sz;i++)cout << s.substr(t[i].st,t[i].len) << '\
      n';
  }//996a3b

};//5bf93b
```

8.8 Manacher

```
struct Manacher{
  vector<int> p[2];
  // p[1][i] = (max odd length palindrome centered at i) / 2 [
     floor division]
  // p[0][i] = same for even, it considers the right center
  // e.g. for s = "abbabba", p[1][3] = 3, p[0][2] = 2
  Manacher(string &s){
    int n = sz(s);
    p[0].resize(n+1);
    p[1].resize(n);
    for (int z=0;z<2;z++){
      for(int i=0,l=0,r=0;i<n;i++){
        int t = r-i+!z;
        if(i<r)p[z][i] = min(t,p[z][l+t]);
        int L=i- p[z][i],R=i+p[z][i]-!z;
        while (L>=1&&R+1<n && s[L-1] == s[R+1])
          p[z][i]++, L--, R++;
        if(R>r)l=L,r=R;
      }
    }
  }

  bool is_palindrome(int l, int r){
    int mid = (l+r+1)/2,len = r-l+1;
    return 2 * p[len % 2][mid] + len % 2 >= len;
  }
};
```

9 Dp

9.1 Convex Hull Trick

```
//Hash: #d1459e
struct CHT {
  vector<ll> m, b;
  int ptr = 0;

  bool bad(int l1, int l2, int l3) {
    return 1.0*(b[l3]-b[l1])*(m[l1]-m[l2])<=1.0*(b[l2]-b[l1])*(m[
     l1]-m[l3]);//(slope dec+query min),(slope inc+query max) // #
     aaf1b6
    return 1.0*(b[l3]-b[l1])*(m[l1]-m[l2]) >1.0*(b[l2]-b[l1])*(m[
     l1]-m[l3]); //(slope dec+query max), (slope inc+query min) //
      #c3130b
  }

  void add(ll _m, ll _b){
    m.push_back(_m);
    b.push_back(_b);
    int s = m.size();
    while(s>=3&&bad(s - 3, s - 2, s - 1)){
      s--;
      m.erase(m.end()-2);
      b.erase(b.end()-2);
    }
  } // #b5b4ed

  ll f(int i, ll x){return m[i] * x + b[i];} // #d77a99

  //(slope dec+query min), (slope inc+query max) -> x increasing
  //(slope dec+query max), (slope inc+query min) -> x decreasing
  ll query(ll x){
    if(ptr >= sz(m))ptr = m.size() - 1;
    while(ptr < sz(m)-1 && f(ptr + 1, x)<f(ptr, x))ptr++;
    return f(ptr, x);
  } // #19a165

  ll bs(int l, int r, ll x){
    int mid = (l + r) / 2;
    if(mid + 1 < m.size() && f(mid + 1, x) < f(mid, x)) return bs(
     mid + 1, r, x); // > for max
    if(mid - 1 >= 0 && f(mid - 1, x) < f(mid, x)) return bs(l, mid
      - 1, x); // > for max
    return f(mid, x);
  } // #59c8d0
};
```

9.2 Knuth Division

```
int N;
// read N and input
int dp[N][N], opt[N][N];

auto C = [&](int i, int j) {
        // Implement cost function C.
};

for (int i=0;i<N;i++){
  opt[i][i] = i;
  // Initialize dp[i][i] according to the problem
}

for (int i = N-2; i >= 0; i--){
  for (int j = i+1; j < N; j++){
    int mn = INT_MAX;
    int cost = C(i, j);
    for (int k = opt[i][j-1]; k <= min(j-1, opt[i+1][j]); k++) {
      if (mn >= dp[i][k] + dp[k+1][j] + cost) {
        opt[i][j] = k;
        mn = dp[i][k] + dp[k+1][j] + cost;
      }
    }
    dp[i][j] = mn;
  }
} // #09b41e
ans = dp[0][N-1];
```

9.3 Li Chao Tree

```
typedef ll ftype;
typedef complex<ftype> point;
#define x real
#define y imag
const ll MOD=1e9+7; //998244353LL;
const ll INF=1LL<<60;
const int maxn = 2e5;
point line[4 * maxn];
ftype dot(point a, point b) {
return (conj(a) * b).x();
}
ftype f(point a, ftype x) {
return dot(a, {x, 1});
}
void add_line(point nw, int v, int l, int r, int
lf, int rf) {
int m = (l + r) / 2;
if(rf < l || r <= lf) return;
if(l < lf || rf < r-1){
add_line(nw, 2 * v, l, m, lf, rf);
add_line(nw, 2 * v + 1, m, r, lf, rf);
return;
}
bool lef = f(nw, l) < f(line[v], l);
bool mid = f(nw, m) < f(line[v], m);
if(!mid) {
swap(line[v], nw);
}
if(r - l == 1) {
return;
} else if(lef != mid) {
add_line(nw, 2 * v, l, m, lf, rf);
} else {
add_line(nw, 2 * v + 1, m, r, lf, rf);
}
}
ftype get(int x, int v, int l, int r) {
int m = (l + r) / 2;
if(r - l == 1) {
return f(line[v], x);
} else if(x < m) {
return max(f(line[v], x), get(x, 2 * v,
l, m));
} else {
return max(f(line[v], x), get(x, 2 * v +
1, m, r));
}
```

10 Geometry 2D

10.1 Point

```
const double eps = 1e-9;
int sign(double x){return (x>eps)-(x<-eps);}

struct PT{
  double x,y;
  PT(){x=0,y=0;}
  PT(double x,double y):x(x),y(y){}
  PT(const PT &p):x(p.x),y(p.y){}
  PT operator+(const PT &a)const{return PT(x+a.x,y+a.y);}
  PT operator-(const PT &a)const{return PT(x-a.x,y-a.y);}
  PT operator*(const double a)const{return PT(x*a,y*a);}
  friend PT operator*(const double &a, const PT &b){return PT(a*b.
     x,a* b.y);}
  PT operator / (const double a)const{return PT(x/a,y/a);}
  bool operator == (PT a)const{return sign(a.x-x)==0 && sign(a.y-y
     )==0;}
  bool operator != (PT a)const{return !(*this==a);}
  bool operator < (PT a) const{return sign(a.x-x)==0?y<a.y:x<a.x;}
  bool operator > (PT a) const{return sign(a.x-x)==0?y>a.y:x>a.x;}
```

```
  double norm(){return sqrt(x*x+y*y);}
  double norm2(){return x*x+y*y;}
  PT perp(){return PT(-y,x);}
  double arg(){return atan2(y,x);}

  // returns a vector with norm r and having same direction
  PT truncate(double r){
    double k = norm();
    if(!sign(k))return *this;
    r/=k;
    return PT(x*r,y*r);
  }
};

istream &operator >> (istream &in,PT &p){return in >> p.x >> p.y;}
ostream &operator << (ostream &out, PT &p){return out << "(" << p.x << "," << p.y << ")";}

inline double dot(PT a,PT b){return a.x*b.x+a.y*b.y;}
inline double dist2(PT a,PT b){return dot(a-b,a-b);}
inline double dist(PT a,PT b){return sqrt(dot(a-b,a-b));}
inline double cross(PT a,PT b){return a.x*b.y-a.y*b.x;}
inline double cross2(PT a,PT b,PT c){return cross(b-a,c-a);}
inline int orientation(PT a,PT b,PT c){return sign(cross(b-a,c-a));}

PT perp(PT a){return PT(-a.y,a.x);}
PT rotateccw90(PT a){return PT(-a.y, a.x);}
PT rotatecw90(PT a){return PT(a.y,-a.x);}
PT rotateccw(PT a,double t){return PT(a.x*cos(t)-a.y*sin(t),a.x*sin(t)+a.y*cos(t));}
PT rotatecw(PT a,double t){return PT(a.x*cos(t)+a.y*sin(t),-a.x*sin(t)+a.y*cos(t));}

double SQ(double x){return x*x;}
double rad_to_deg(double r){return (r*180.0/PI);}
double deg_to_rad(double d){return (d*PI/180.0);}
double get_angle(PT a, PT b){
  double costheta = dot(a,b)/a.norm()/b.norm();
  return acos(max((double)-1.0, min((double)1.0, costheta)));
}

// does point p lie in angle <bac
bool is_point_in_angle(PT b,PT a,PT c,PT p){
  assert(orientation(a,b,c)!= 0);
  if(orientation(a,c,b)<0)swap(b,c);
  return orientation(a,c,p)>=0 && orientation(a,b,p)<=0;
}
bool half(PT p) {
    return p.y > 0.0 || (p.y == 0.0 && p.x < 0.0);
}

// sort points in counterclockwise
void polar_sort(vector<PT> &v){
  sort(all(v),[](PT a,PT b){
    return make_tuple(half(a),0.0,a.norm2())<make_tuple(half(b),cross(a,b),b.norm2());
  });
}

//sort points in counterclockwise with respect to point o
void polar_sort(vector<PT> &v, PT o){
  sort(all(v),[&](PT a,PT b){
    return make_tuple(half(a-o),0.0,(a-o).norm2())<make_tuple(half(b-o),cross(a-o,b-o),(b-o).norm2());
  });
}
```

10.2 Line

```
struct line{
  PT a, b; // goes through points a and b
  PT v; double c;  //line form: direction vec [cross] (x, y) = c
  line() {}

  //direction vector v and offset c
  line(PT v, double c) : v(v), c(c){
    auto p = get_points();
    a = p.first; b = p.second;
  }

  // equation ax + by + c = 0
  line(double _a, double _b, double _c):v({_b, -_a}),c(-_c){
    auto p = get_points();
    a = p.first; b = p.second;
  }

  // goes through points p and q
  line(PT p, PT q) : v(q - p), c(cross(v, p)), a(p), b(q) {}

  pair<PT, PT> get_points(){//extract any two points from this line
    PT p,q; double a = -v.y, b = v.x; // ax + by = c
    if(sign(a) == 0){
      p = PT(0, c / b);
      q = PT(1, c / b);
    }else if (sign(b) == 0){
        p = PT(c/a,0);
        q = PT(c/a,1);
    }else{
      p = PT(0, c/b);
      q = PT(1, (c-a)/b);
    }
    return {p, q};
  }

  //ax + by + c = 0
  array<double, 3> get_abc(){
    double a = -v.y, b = v.x;
    return {a, b, -c};
  }

  // 1 if on the left, -1 if on the right, 0 if on the line
  int side(PT p){return sign(cross(v, p) - c);}

  // line that is perpendicular to this and goes through point p
  line perpendicular_through(PT p) {return {p, p + perp(v)};}
  // translate the line by vector t i.e. shifting it by vector t
  line translate(PT t) { return {v, c + cross(v, t)}; }
  // compare two points by their orthogonal projection on this line
  // a projection point comes before another if it comes first according to vector v
  bool cmp_by_projection(PT p, PT q) { return dot(v, p) < dot(v, q); }
  line shift_left(double d) {
    PT z = v.perp().truncate(d);
    return line(a + z, b + z);
  }
};

// find a point from a through b with distance d
PT point_along_line(PT a, PT b, double d){
  assert(a != b);
  return a + (((b - a) / (b - a).norm()) * d);
}
// projection point c onto line through a and b  assuming a != b
PT project_from_point_to_line(PT a, PT b, PT c){
  return a + (b - a) * dot(c - a, b - a)/(b - a).norm2();
}
// reflection point c onto line through a and b  assuming a != b
PT reflection_from_point_to_line(PT a, PT b, PT c){
  PT p = project_from_point_to_line(a,b,c);
  return p + p - c;
}
// minimum distance from point c to line through a and b
double dist_from_point_to_line(PT a, PT b, PT c){
  return fabs(cross(b - a, c - a)/(b - a).norm());
}
// returns true if  point p is on line segment ab
bool is_point_on_seg(PT a, PT b, PT p){
  if(fabs(cross(p - b, a - b)) < eps) {
    if (p.x < min(a.x, b.x) - eps || p.x > max(a.x, b.x) + eps)
      return false;
    if (p.y < min(a.y, b.y) - eps || p.y > max(a.y, b.y) + eps)
      return false;
    return true;
  }
  return false;
}
// minimum distance point from point c to segment ab that lies on segment ab
PT project_from_point_to_seg(PT a, PT b, PT c){
  double r = dist2(a, b);
  if (sign(r) == 0) return a;
  r = dot(c - a, b - a) / r;
  if(r<0)return a;
  if(r>1)return b;
  return a + (b - a) * r;
}
// minimum distance from point c to segment ab
double dist_from_point_to_seg(PT a, PT b, PT c){
  return dist(c, project_from_point_to_seg(a, b, c));
}
// 0 if not parallel, 1 if parallel, 2 if collinear
int is_parallel(PT a, PT b, PT c, PT d){
  double k = fabs(cross(b - a, d - c));
  if(k < eps){
    if(fabs(cross(a - b, a - c)) < eps && fabs(cross(c - d, c - a)) < eps) return 2;
    else return 1;
  }else return 0;
}
// check if two lines are same
bool are_lines_same(PT a, PT b, PT c, PT d){
  if(fabs(cross(a - c, c - d)) < eps && fabs(cross(b - c, c - d)) < eps) return true;
  return false;
}
// bisector vector of <abc
PT angle_bisector(PT &a, PT &b, PT &c){
  PT p = a - b, q = c - b;
  return p + q * sqrt(dot(p, p) / dot(q, q));
}
// 1 if point is ccw to the line, 2 if point is cw to the line, 3 if point is on the line
int point_line_relation(PT a, PT b, PT p){
  int c = sign(cross(p - a, b - a));
  if (c < 0) return 1;
  if (c > 0) return 2;
  return 3;
}
```

```
// intersection point between ab and cd assuming unique intersection exists
bool line_line_intersection(PT a, PT b, PT c, PT d, PT &ans){
  double a1 = a.y - b.y, b1 = b.x - a.x, c1 = cross(a, b);
  double a2 = c.y - d.y, b2 = d.x - c.x, c2 = cross(c, d);
  double det = a1 * b2 - a2 * b1;
  if(det == 0) return 0;
  ans = PT((b1 * c2 - b2 * c1) / det, (c1 * a2 - a1 * c2) / det);
  return 1;
}
// intersection point between segment ab and segment cd assuming unique intersection exists
bool seg_seg_intersection(PT a, PT b, PT c, PT d, PT &ans){
  double oa = cross2(c, d, a), ob = cross2(c, d, b);
  double oc = cross2(a, b, c), od = cross2(a, b, d);
  if(oa * ob < 0 && oc * od < 0){
    ans = (a * ob - b * oa) / (ob - oa);
    return 1;
  }
  else return 0;
}
// intersection point between segment ab and segment cd assuming unique intersection may not exists
// se.size()==0 means no intersection
// se.size()==1 means one intersection
// se.size()==2 means range intersection
set<PT> seg_seg_intersection_inside(PT a,  PT b,  PT c,  PT d){
  PT ans;
  if (seg_seg_intersection(a, b, c, d, ans)) return {ans};
  set<PT> se;
  if (is_point_on_seg(c, d, a)) se.insert(a);
  if (is_point_on_seg(c, d, b)) se.insert(b);
  if (is_point_on_seg(a, b, c)) se.insert(c);
  if (is_point_on_seg(a, b, d)) se.insert(d);
  return se;
}
// intersection  between segment ab and line cd
// 0 if do not intersect, 1 if proper intersect, 2 if segment intersect
int seg_line_relation(PT a, PT b, PT c, PT d){
  double p = cross2(c, d, a);
  double q = cross2(c, d, b);
  if(sign(p) == 0 && sign(q) == 0)return 2;
  else if(p * q < 0)return 1;
  else return 0;
}
// intersection between segament ab and line cd assuming unique intersection exists
bool seg_line_intersection(PT a, PT b, PT c, PT d, PT &ans){
  bool k = seg_line_relation(a, b, c, d);
  assert(k != 2);
  if(k)line_line_intersection(a, b, c, d, ans);
  return k;
}
// minimum distance from segment ab to segment cd
double dist_from_seg_to_seg(PT a, PT b, PT c, PT d){
  PT dummy;
  if(seg_seg_intersection(a, b, c, d, dummy))return 0.0;
  else return min({dist_from_point_to_seg(a, b, c), dist_from_point_to_seg(a, b, d),
      dist_from_point_to_seg(c, d, a), dist_from_point_to_seg(c, d, b)});
}
// minimum distance from point c to ray (starting point a and direction vector b)
double dist_from_point_to_ray(PT a, PT b, PT c){
  b = a + b;
  double r = dot(c - a, b - a);
  if (r < 0.0) return dist(c, a);
  return dist_from_point_to_line(a, b, c);
}
// starting point as and direction vector ad
bool ray_ray_intersection(PT as, PT ad, PT bs, PT bd){
  double dx = bs.x - as.x, dy = bs.y - as.y;
  double det = bd.x * ad.y - bd.y * ad.x;
  if (fabs(det) < eps) return 0;
  double u = (dy * bd.x - dx * bd.y) / det;
  double v = (dy * ad.x - dx * ad.y) / det;
  if(sign(u) >= 0 && sign(v) >= 0) return 1;
  else return 0;
}
double ray_ray_distance(PT as, PT ad, PT bs, PT bd){
  if(ray_ray_intersection(as, ad, bs, bd)) return 0.0;
  double ans = dist_from_point_to_ray(as, ad, bs);
  ans = min(ans, dist_from_point_to_ray(bs, bd, as));
  return ans;
}
```

10.3 Circle

```
struct circle{
  PT p; double r;
  circle() {}
  circle(PT _p, double _r): p(_p), r(_r) {};
  // center (x, y) and radius r
  circle(double x, double y, double _r): p(PT(x, y)), r(_r) {};
  // circumcircle of a triangle
  // the three points must be unique
  circle(PT a, PT b, PT c){
    b = (a + b) * 0.5;
    c = (a + c) * 0.5;
    line_line_intersection(b, b + rotatecw90(a - b), c, c + rotatecw90(a - c), p);
    r = dist(a, p);
  }
  // inscribed circle of a triangle
  // pass a bool just to differentiate from circumcircle
  circle(PT a, PT b, PT c, bool t){
    line u, v;
    double m = atan2(b.y - a.y, b.x - a.x), n = atan2(c.y - a.y, c.x - a.x);
    u.a = a;
    u.b = u.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
    v.a = b;
    m = atan2(a.y - b.y, a.x - b.x), n = atan2(c.y - b.y, c.x - b.x);
    v.b = v.a + (PT(cos((n + m)/2.0), sin((n + m)/2.0)));
    line_line_intersection(u.a, u.b, v.a, v.b, p);
    r = dist_from_point_to_seg(a, b, p);
  }
  bool operator == (circle v) { return p == v.p && sign(r - v.r) == 0; }
  double area() { return PI * r * r; }
  double circumference() { return 2.0 * PI * r; }
};

//0 if outside, 1 if on circumference, 2 if inside circle
int circle_point_relation(PT p, double r, PT b){
  double d = dist(p, b);
  if(sign(d - r) < 0) return 2;
  if (sign(d - r) == 0) return 1;
  return 0;
}
// 0 if outside, 1 if on circumference, 2 if inside circle
int circle_line_relation(PT p, double r, PT a, PT b){
  double d = dist_from_point_to_line(a, b, p);
  if (sign(d - r) < 0) return 2;
  if (sign(d - r) == 0) return 1;
  return 0;
}
//compute intersection of line through points a and b with
//circle centered at c with radius r > 0
vector<PT> circle_line_intersection(PT c, double r, PT a, PT b){
  vector<PT> ret;
  b = b - a; a = a - c;
  double A = dot(b, b), B = dot(a, b);
  double C = dot(a, a) - r * r, D = B * B - A * C;
  if (D < -eps) return ret;
  ret.push_back(c + a + b * (-B + sqrt(D + eps)) / A);
  if (D > eps) ret.push_back(c + a + b * (-B - sqrt(D)) / A);
  return ret;
}
//5 - outside and do not intersect
//4 - intersect outside in one point
//3 - intersect in 2 points
//2 - intersect inside in one point
//1 - inside and do not intersect
int circle_circle_relation(PT a, double r, PT b, double R){
  double d = dist(a, b);
  if (sign(d - r - R) > 0)  return 5;
  if (sign(d - r - R) == 0) return 4;
  double l = fabs(r - R);
  if (sign(d - r - R) < 0 && sign(d - l) > 0) return 3;
  if (sign(d - l) == 0) return 2;
  if (sign(d - l) < 0) return 1;
  assert(0); return -1;
}
vector<PT> circle_circle_intersection(PT a, double r, PT b, double R){
  if (a == b && sign(r - R) == 0) return {PT(1e18, 1e18)};
  vector<PT> ret;
  double d = sqrt(dist2(a,  b));
  if (d > r + R || d + min(r,  R) < max(r,  R)) return ret;
  double x = (d * d - R * R + r * r) / (2 * d);
  double y = sqrt(r * r - x * x);
  PT v = (b - a) / d;
  ret.push_back(a + v * x  +  rotateccw90(v) * y);
  if (y > 0) ret.push_back(a + v * x - rotateccw90(v) * y);
  return ret;
}
// returns two circle c1, c2 through points a, b and of radius r
// 0 if there is no such circle, 1 if one circle, 2 if two circle
int get_circle(PT a, PT b, double r, circle &c1, circle &c2){
  vector<PT> v = circle_circle_intersection(a, r, b, r);
  int t = v.size();
  if(!t) return 0;
  c1.p = v[0], c1.r = r;
  if(t == 2) c2.p = v[1], c2.r = r;
  return t;
}
// returns area of intersection between two circles
double circle_circle_area(PT a, double r1, PT b, double r2){
  double d = (a - b).norm();
  if(r1 + r2 < d + eps) return 0;
  if(r1 + d < r2 + eps) return PI * r1 * r1;
  if(r2 + d < r1 + eps) return PI * r2 * r2;
  double theta_1 = acos((r1 * r1 + d * d - r2 * r2) / (2 * r1 * d)),
    theta_2 = acos((r2 * r2 + d * d - r1 * r1)/(2 * r2 * d));
  return r1 * r1 * (theta_1 - sin(2 * theta_1)/2.) + r2 * r2 * (theta_2 - sin(2 * theta_2)/2.);
}
```

10.4 Tangent

```
// tangent lines from point q to the circle
int tangent_lines_from_point(PT p, double r, PT q, line &u, line &v){
  int x = sign(dist2(p, q) - r * r);
  if (x < 0) return 0; // point in cricle
  if (x == 0){ // point on circle
    u = line(q, q + rotateccw90(q - p));
    v = u;
    return 1;
  }

  double d = dist(p, q);
  double l = r * r / d;
  double h = sqrt(r * r - l * l);
  u = line(q, p + ((q - p).truncate(l) + (rotateccw90(q - p).truncate(h))));
  v = line(q, p + ((q - p).truncate(l) + (rotatecw90(q - p).truncate(h))));
  return 2;
}
// returns outer tangents line of two circles
// if inner == 1 it returns inner tangent lines
int tangents_lines_from_circle(PT c1, double r1, PT c2, double r2, bool inner, line &u, line &v){
  if(inner) r2 = -r2;
  PT d = c2 - c1;
  double dr = r1 - r2, d2 = d.norm2(), h2 = d2 - dr * dr;
  if(d2 == 0 || h2 < 0){
    assert(h2 != 0);
    return 0;
  }
  vector<pair<PT, PT>>out;
  for (int tmp: {- 1, 1}){
    PT v = (d * dr + rotateccw90(d) * sqrt(h2) * tmp) / d2;
    out.push_back({c1 + v * r1, c2 + v * r2});
  }
  u = line(out[0].first, out[0].second);
  if(out.size() == 2) v = line(out[1].first, out[1].second);
  return 1 + (h2 > 0);
}
// returns two circle c1, c2 which is tangent to line u,  goes through
// point q and has radius r1; 0 for no circle, 1 if c1 = c2 , 2 if c1 != c2
int get_circle(line u, PT q, double r1, circle &c1, circle &c2){
  double d = dist_from_point_to_line(u.a, u.b, q);
  if (sign(d - r1 * 2.0) > 0) return 0;
  if (sign(d) == 0) {
    cout << u.v.x << ' ' << u.v.y << '\n';
    c1.p = q + rotateccw90(u.v).truncate(r1);
    c2.p = q + rotatecw90(u.v).truncate(r1);
    c1.r = c2.r = r1;
    return 2;
  }
  line u1 = line(u.a + rotateccw90(u.v).truncate(r1), u.b + rotateccw90(u.v).truncate(r1));
  line u2 = line(u.a + rotatecw90(u.v).truncate(r1), u.b + rotatecw90(u.v).truncate(r1));
  circle cc = circle(q, r1);
  PT p1, p2; vector<PT> v;
  v = circle_line_intersection(q, r1, u1.a, u1.b);
  if(!v.size()) v = circle_line_intersection(q, r1, u2.a, u2.b);
  v.push_back(v[0]);
  p1 = v[0], p2 = v[1];
  c1 = circle(p1, r1);
  if(p1 == p2){
    c2 = c1;
    return 1;
  }
  c2 = circle(p2, r1);
  return 2;
}
```

10.5 Convex Hull

```
vector<PT> convex_hull(vector<PT> &p){
  if (p.size() <= 1) return p;
  vector<PT> v = p;
  sort(v.begin(), v.end());
  vector<PT> up, dn;
  for(auto& p:v){
    while(up.size() > 1 && orientation(up[up.size() - 2], up.back(), p) >= 0)
      up.pop_back();

    while(dn.size() > 1 && orientation(dn[dn.size() - 2], dn.back(), p) <= 0)
      dn.pop_back();

    up.push_back(p);
    dn.push_back(p);
  }
  v = dn;
  if (v.size() > 1) v.pop_back();
  reverse(up.begin(), up.end());
  up.pop_back();
  for (auto& p : up)
    v.push_back(p);

  if (v.size() == 2 && v[0] == v[1]) v.pop_back();
  return v;
}
```

10.6 Polygon

```
double area_of_triangle(PT a, PT b, PT c) {
    return fabs(cross(b - a, c - a) * 0.5);
}
// -1 if strictly inside, 0 if on the polygon, 1 if strictly outside
int is_point_in_triangle(PT a, PT b, PT c, PT p) {
    if (sign(cross(b - a,c - a)) < 0) swap(b, c);
    int c1 = sign(cross(b - a,p - a));
    int c2 = sign(cross(c - b,p - b));
    int c3 = sign(cross(a - c,p - c));
    if (c1<0 || c2<0 || c3 < 0) return 1;
    if (c1 + c2 + c3 != 3) return 0;
    return -1;
}
double perimeter(vector<PT> &p) {
    double ans=0; int n = p.size();
    for (int i = 0; i < n; i++) ans += dist(p[i], p[(i + 1) % n]);
    return ans;
}
double area(vector<PT> &p) {
    double ans = 0; int n = p.size();
    for (int i = 0; i < n; i++) ans += cross(p[i], p[(i + 1) % n]);
    return fabs(ans) * 0.5;
}
```

11 Miscellaneous

11.1 Rnd template

```
mt19937_64 rnd(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<ll> dist(1ll,INF64);
// dist(rnd)/3 , rnd()/3
```